FULL PROJECT REPORT · STATE ON 7 OCT 2026, 20:15

AEGIS-UAV: the complete record of what was built, tested and found

A drone has a camera and must detect objects in every frame. It can run the detector on its own small computer, send the image to a more powerful computer nearby (another drone or a ground station), or split the neural network between them. This project measures which option works best, first in simulation and then on real hardware, and builds a controller that switches between options as conditions change.

Labels used throughout: **VALIDATED** measured, repeated and cross-checked; **PENDING** built, but the decisive run has not finished; **INVALIDATED** results found to be affected by a fault and set aside; **INCONCLUSIVE** measured, but the difference is within the confidence interval; the same color marks **NOT SUPPORTED**, a claim the data did not bear out.

Contents

1. Where things stand right now
2. The technology, explained
3. Phase A: the split-inference toolkit and paper
4. Phase B: checking the simulator
5. Phase C: the hardware testbed
6. Device and link measurements
7. Every placement of the model
8. Corrections to Hardware Testing.pdf
9. Adaptive controller v1
10. Adaptive controller v2: multi-server scheduling
11. Literature check and novelty (re-checked)
12. Phase D: task-level test on drone video
13. Phase E: swarm simulation calibrated to the hardware
14. Phase F: motion forecasting of stale detections
15. Phase G: energy, power and heat
16. What it all means: the argument for the paper
17. Faults found and fixed
18. Code status and reproducibility
19. What is left
20. Where everything is
21. Glossary

PART 1

Where things stand right now

> **Current state: the real-world Wi-Fi campaign, motion forecasting and swarm coordination are finished: the mesh campaign comes next**

- The task-level test on labeled drone video ran on the current shared Wi-Fi: 9 policies × 4 scenarios × 3 repetitions, 108 runs, 01:33–06:21 on 6 October, with no outages or failed runs (Part 12).
- Motion forecasting, tuned on training videos only, raised streaming AP for every policy by 32–67 % (median) on those 108 runs. The camera-aligned forecaster v2 roughly doubles it (+80 % to +156 %) and beats v1 in all 144 runs (Part 14).
- Swarm-level coordination was added to the calibrated swarm simulation and confirmed on held-out seeds (Part 13).
- v3 rebuilt on the forecasting decay curve ("v3 + forecasting") ran in a paired Wi-Fi campaign against v2 and the old v3: 36 runs, 06:52–08:29, no outages (Part 14).
- Done today (6 October): 80-run campaign with v3 + forecasting v2 and the live forecaster; 24-run two-camera swarm campaign; simulator-vs-hardware check; coordinator with radio link health.
- Code: all 118 project tests pass, the simulator's default behaviour is bit-for-bit unchanged, and nothing is running. Nothing is committed to git yet (Part 18).
- The controlled version of the campaign, planned for the AegisSwarm mesh, has 3 of its 180 runs.
- The plan is to move the Pi, the Jetson and the laptop onto the AegisSwarm ad-hoc (IBSS) mesh, a private channel with real drone-to-drone links. The laptop keeps internet through an Ethernet cable so Claude stays connected.
- Waiting on you: the Ethernet cable connected, and AegisSwarm's channel, band, IP range and whether batman-adv runs on top.

Results so far

Splitting the network never beats sending a JPEG

Shown in simulation, in theory (a closed-form bound), on 19 detectors, and now on real hardware: across 1,150 placements over the Pi, the Jetson and the laptop, and in live runs. `VALIDATED`

An adaptive multi-server scheduler beats every fixed strategy

59–62 % more accuracy-weighted on-time detections than the best fixed strategy on impaired links, 19 % on a clean link, and best in all 8 paired comparisons. This is scored with COCO accuracy weights, a proxy metric. `VALIDATED`

Perception survives a total blackout

The scheduler switches to YOLOv8n on the Pi and keeps about 10 on-time detections per second. Availability (newest detection under 300 ms old) was 0.999 in every scenario. `VALIDATED`

Aerial detections lose 58 % of their accuracy in 100 ms

A new measurement on VisDrone video: a 300 ms deadline badly overstates real perception quality on drones. **VALIDATED**

In a simulated swarm, adaptive degrades gracefully where fixed offload collapses

With measured Pi and Wi-Fi numbers, fixed JPEG falls to 5.5 frames/s per drone at 5 drones and 0 at 10. Adaptive holds 15.0 at 5 and 8.3 at 10. It is best or tied in 78 of 95 comparisons, and at most 8.3 % behind in the rest. Large swarms need coordination between drones. **VALIDATED** (simulation)

On real accuracy over real Wi-Fi, the multi-server controller v2 is the best policy

Across 108 runs scored by streaming AP on VisDrone video, v2 beats every other policy on average in paired comparisons: by 4.5 % over all servers without adaptation, 14 % over single-path rate adaptation, 37 % over the best fixed server, and 75–201 % over rule-based fallback, the Jetson and onboard. The margin is statistically significant (95 % interval above zero) against rule-based fallback, the Jetson and onboard, but not yet against the others. **VALIDATED** (Wi-Fi)

Motion forecasting recovers much of the staleness loss

Moving each old box forward by its estimated velocity raised streaming AP for all 9 policies, by a median of 32–67 %, with no extra traffic. On training videos a result shown 100 ms late keeps 77 % of fresh accuracy (41 % without), and 47 % at 267 ms (7 % without). **VALIDATED**

Energy-aware control is both more efficient and more accurate

Measured on both boards and applied to every campaign (Part 15): earlier controllers ran the fanless Pi at 4.2–5.3 W, above the about 3.6 W it can sustain without throttling. Onboard inference was 40–54 % of the power for 3–5 % of the accuracy. Controller v4 prices energy against a measured power budget. It cuts measured Pi power by about 1 W (−25 %) and time in the throttling region from 76 % to 17 %, and it is significantly

more accurate

than every other controller: +0.0074 ± 0.0028 sAP over v3 + forecasting v2, 17 of 20 pairs. The cost is freshness (87 % against 99 %). **VALIDATED** (100 runs, measured energy)

Measuring the camera's motion doubles streaming accuracy

Forecaster v2 tracks the background between frames (2.7 ms per frame on the Pi), carries old boxes with the camera's motion, then adds each object's own velocity. Kept accuracy at 267 ms rises to 60 % (v1 47 %, none 7 %). On all 144 real runs it beats v1 every time (median +51 %) and roughly doubles streaming AP compared with no forecasting. **VALIDATED**

roughly doubles streaming AP compared with no forecasting. VALIDATED

A controller built for the forecaster it runs with wins, significantly

v3 rebuilt on the camera-aligned forecaster's decay curve, with the forecaster running live on the Pi, beats v2 by +0.0045 ± 0.0035 sAP over 20 paired runs (median +8.9 %). It is best in 3 of 4 scenarios, including the clean link where fixed laptop YOLOv8m had won before, and keeps a fresh detection 99.5–99.8 % of the time. **VALIDATED** (Wi-Fi, 5 repetitions)

Swarm coordination fixes congestion collapse and starvation

In the calibrated simulation, a ground-side admission rule turns the collapse of 20 Pi drones into about 4.1 detections/s each, raises Jetson-class swarms on Wi-Fi by up to 27 %, and stops the router from starving drones (worst drone 0 → 7.65 frames/s at 10 drones). With each radio's own link health added, its worst loss shrinks from −7.4 % to −1.9 %. Same results on held-out seeds. **VALIDATED** (simulation)

A real two-camera swarm: sharing costs, adaptation pays

With the Pi and the Jetson both as camera drones sharing the laptop, the Pi loses 11–19 % of its streaming accuracy to sharing (significant). Adaptive control beats fixed offloading for the swarm in 6 of 6 pairs. The live coordinator works but rarely acts with two cameras, as the simulation predicted. **VALIDATED** · **INCONCLUSIVE** (coordinator)

The simulator matches the hardware, except after blackouts

Rebuilt from 44 real runs (528 phases) with nothing fitted, it predicts on-time results per phase with correlation 0.91 (average error 2.0/s) once the controller's one-frame-in-flight rule is modelled. After a blackout the real link stays slow (latency 389 ms against 71 ms), which the simulator does not capture. **VALIDATED** (with that limit)

v3 rebuilt with forecasting keeps a fresh detection 99–100 % of the time

In a paired 36-run campaign it held a detection under 300 ms old 99.0–99.9 % of the time in every scenario, against 75–92 % for v2 under impairment. Its accuracy is level with v2: the difference is not significant, it is ahead on the mean and behind in 8 of 12 pairs. **VALIDATED** (availability) · **INCONCLUSIVE** (accuracy)

The original staleness-aware controller v3 did not win

v3 beat the fixed strategies, but trailed v2 by a median 14 % (10 of 12 pairs). Its objective was built on the steep decay curve measured without forecasting, which makes it over-favour fast results. Rebuilt on the forecasting curve, it became "v3 + forecasting" (above). **NOT SUPPORTED** (original v3)

PART 2

The technology, explained

The detector: YOLOv8

YOLOv8 is a family of real-time object detectors (n, s, m, l, x, from small to large). YOLOv8s, the main model here, has 23 layers numbered 0 to 22. Layers 0–9 form the backbone, which extracts features. Layers 10–21 form the neck, which combines them at three scales. Layer 22 is the detection head. After the network, non-maximum suppression (NMS) removes duplicate boxes.

Split computing

Split computing runs the first layers on the drone and sends the intermediate result to a server, which runs the rest. A cut `k` means layers 0 to k run on the drone. Some later layers also need outputs from earlier ones (skip connections), so a cut can require sending several tensors. At cut `k4` this is one tensor of 128×80×80 numbers; at `k9` it is three. To make that payload small, each number is quantized to 8 bits (INT8) and compressed losslessly with zstd.

The alternative: image offload

Instead of splitting, the drone can compress the camera frame as a JPEG and send it. The server runs the whole detector. JPEG quality 90 (q90) is near-lossless for detection; q50 is smaller but costs some accuracy.

The three tiers

TIER	DEVICE HERE	ROLE
Sense (camera drone)	Raspberry Pi 5, 4-core ARM CPU, no GPU	Captures at 30 fps; can run small models onboard
Edge (mothership drone)	NVIDIA Jetson Orin Nano, 1024-core GPU, 15 W mode	A flying GPU near the camera drone
Cloud (ground station)	Laptop with RTX 4050 GPU	The most compute, the farthest away

Measuring quality

- **mAP@[.5:.95]:** the standard detection accuracy, averaged over overlap thresholds from 0.5 to 0.95.
- **On-time frames/s:** detections that come back within a deadline (300 ms in early tests).

- **Streaming AP (sAP):** at every camera frame, whatever detections the drone holds, possibly from an older frame, are scored against what the camera sees now. Latency, dropped frames and accuracy count together. It is the standard real-time perception metric (Li, Wang and Ramanan, ECCV 2020).
- **Availability:** the share of time the drone holds a detection less than 300 ms old.
- **Age of information (AoI):** the age of the newest detection the drone holds.

The three controller versions

- **v1** picks one strategy at a time (onboard, or one server with one setting), scoring each by expected on-time results per second, and switches with hysteresis.
- **v2** uses all three computers at once. For every frame it picks a computer that is free and the best setting for it: model, image size and JPEG quality. Its score is accuracy × probability of arriving on time ÷ time the computer is busy. "A_full" is v2 with COCO accuracy, "A_full_aer" v2 with aerial accuracy.
- **v3** ("A_stream") keeps v2's dispatch but scores a result by the accuracy it will actually provide while it is held: accuracy × probability on time × the average staleness factor over the time it is shown. The staleness factor comes from the decay curve measured on training videos.

Staleness and motion forecasting

A detection describes where objects were when the frame was captured. By the time it is shown, the drone and the objects have moved, so its boxes no longer line up with the scene. On aerial video this decay is fast: with no correction, a detection 100 ms old keeps about 40 % of its accuracy. Motion forecasting estimates each box's velocity from successive results and moves it forward by velocity × age before it is used. It only uses results that have already arrived, so a drone can run it live.

Swarm coordination

When many drones share one network, each one deciding alone sees only its own link. Together they can send more than the network carries: every frame then arrives late, or a router serves some drones and starves the rest. A coordinator on the ground side decides how many drones may offload at once and which settings fit. Drones take turns, and the others process onboard or wait. This is admission control, the same idea networks use to keep calls from overloading a link.

Statistics used

- **95 % interval (±):** the range in which the true mean lies with 95 % confidence, from a t-distribution over repetitions. With 3 repetitions it is wide.
- **Paired comparison:** two policies run under the same scenario and repetition are compared directly. The mean of those differences and its interval tell whether one is reliably better: the interval must not cross zero.
- **Held-out data or seeds:** settings are chosen on one set (training videos, tuning seeds) and results

are reported on a different set, so nothing is fitted to the test.

Network emulation

To create repeatable bad networks, the Linux tool `tc netem` on the Pi limits rate (e.g. 3 Mbit/s) or drops every packet (blackout). The rules apply only to the experiment's data ports, so the SSH control connection is never cut.

PART 3

Phase A: the split-inference toolkit and paper

The first phase built AEGIS-UAV, an exact split-inference toolkit for YOLOv8, and a full simulation study written up as a paper (`paper/main.tex`).

RESULT	VALUE	STATUS
Splitting is exact at every cut, for 19 detectors	Bit-identical outputs	**VALIDATED**
INT8 quantization of the crossing tensors	At most 0.04 mAP points lost at any cut (COCO val2017, 5,000 images)	**VALIDATED**
Smallest split payload	310.7 KiB at k4 (INT8+zstd), 2.4× a JPEG q95 frame, which loses only 0.21 points	**VALIDATED**
Across 19 detectors from 5 YOLO families	Smallest split is 2.0–11.5× a JPEG q90 frame	**VALIDATED**
System simulation (16 channel × hardware settings)	JPEG is the best fixed strategy in all 16; splitting beats onboard in 10	**VALIDATED**
Monte Carlo (1,000 single + 300 swarm random systems)	A split beats JPEG by at most 0.47 %; the adaptive policy falls below JPEG in 112 of 1,300 systems, by at most 7 %	**VALIDATED**
Closed-form theory (Propositions 1–5)	Quantization bound, offload dominance, break-even, swarm capacity, endurance; 40 checks pass	**VALIDATED**

PART 4

Phase B: checking the simulator

The paper's system results come from a Python simulator, so it was checked against independent tools.

CHECK	TOOL	RESULT
C1 Channel statistics	Theory	Rician K estimated 8.99/3.03/0.10 dB for 9/3/0 dB; Clarke autocorrelation within 0.012
V1 Drone-to-drone Wi-Fi	ns-3 802.11ax	Our model is conservative: ns-3 gives 1.4–3.2× more throughput
V2 / O1 Cellular uplink	ns-3 LTE, Simu5G 5G NR	Our mapping is pessimistic at high signal and optimistic at low; conclusions re-checked with the measured curves
V3/V4 Frame streaming	ns-3, real TCP	44 settings, correlation 0.98, same best strategy in 8 of 8
Calibrated rerun	ns-3 link curves	JPEG still best in 16 of 16; adaptation gains up to 1.54×
O2 Cell sharing	Simu5G	Fair sharing confirmed up to 5 drones; at 10+ the real scheduler serves 7 and starves the rest
O4 Wi-Fi range per rate	INET (OMNeT++)	Matches our model for 7 of 9 rates
O5 Two-hop relay	ns-3	Our relay model is conservative: the real path is 1.3–3.1× faster
V5 Real network stack	Linux TCP + netem	Within 0.9 frames/s in 6 settings

Status: **VALIDATED** . Every conclusion of the paper survived the calibrated reruns. Details: `docs/assumptions_and_validation.md` .

PART 5

Phase C: the hardware testbed

Setup work

- **Access:** SSH to both boards, with passwords passed only at run time and never stored. The Jetson's host-key change was verified against the fingerprint you read on the device.
- **Jetson:** it had been reflashed without CUDA. Installed JetPack 7.2.1 (CUDA 13.2, cuDNN 9.20, TensorRT 10.16) and CUDA PyTorch 2.14.1. That build has no native Orin kernels and runs the sm_80 ones, so detections were checked against the CPU: identical on 10 images, boxes within 0.02 px. Clocks are locked with `jetson_clocks` for every run, because the GPU otherwise down-clocks at low load.
- **Pi:** two findings. Importing ultralytics silently caps PyTorch at one thread (950 ms per frame against 576 ms with four), so the scripts reset it. The Pi has no cooler and throttles from 2.4 to 1.5 GHz at

85–89 °C.

- **Laptop:** the code runs in WSL2 behind Windows' address translation, so the boards cannot open connections to it. Every protocol therefore has the laptop connect out.

Tools written (`validation/hardware/`)

SCRIPT	PURPOSE
`hwbench.py`	Times every layer and all 276 model segments on each device, plus INT8/FP16 encoding at every cut, JPEG, and board power (Jetson INA3221 sensor, Pi PMIC rails, laptop nvidia-smi)
`netbench.py`	Link round-trip time, bulk throughput and per-frame transfer time
`placement.py`	Combines the measurements into all 1,150 placements
`cascade_accuracy.py`	Accuracy when data is encoded twice (two hops)
`pipeline_node.py`	A live distributed pipeline across the three devices
`adaptive.py` , `adaptive2.py`	Adaptive controllers v1, v2 and v3, and the workers
`forecast.py`	Motion forecaster (v1 velocity, v2 camera-aligned): train detections, two tuning stages, decay curves
`sim_vs_real.py`	Rebuilds real runs in the simulator and compares predicted with measured on-time rates
`coordinator.py` , `tools/arun2.sh` , `tools/campaign_swarm.sh` , `summarize_swarm.py`	Two-camera swarm: live coordinator, run script, campaign, scoring
`egomotion.py`	Camera motion between frames: optical flow and RANSAC similarity transform, chained per sequence
`swarm_calibrated.py`	Swarm simulation with the measured hardware numbers, with and without coordination
`stream_ap.py` , `summarize_*.py`	Streaming-AP scoring and campaign statistics
`tools/`	Campaign harness: SSH helpers, run script with stale-process cleanup and clock locking, resumable campaigns with an outage guard

PART 3

Device and link measurements

DEVICE	YOLOV8S FULL	LAYERS 0–4	POWER IDLE → ACTIVE	ENERGY PER FRAME
Pi 5, 4 threads, cool	576 ms	210 ms	2.6 → 4.8 W	1.3 J
Pi 5, sustained (1.5 GHz)	758 ms	252 ms	2.0 → 4.7 W	2.0 J
Pi 5, 1 thread	950 ms	325 ms	—	—
Jetson Orin Nano FP16	22.7 ms	7.4 ms	3.2 → 10.5 W	0.17 J
Jetson Orin Nano FP32	41.7 ms	13.0 ms	3.3 → 9.7 W	0.27 J
Laptop RTX 4050 FP16	5.5 ms	1.0 ms	2.7 → 49 W	—

Pi power is the sum of its PMIC rails, which excludes 5 V conversion losses, so it is a lower bound. The Pi's Wi-Fi rail rises by 0.31 W while transmitting, against 2.7 W for inference.

Small models and resolutions (per frame, including pre- and post-processing)

DEVICE	VARIANT	TIME
Pi (4 threads)	YOLOv8n @ 320 / 416 / 640	72 / 107 / 235 ms
Pi (4 threads)	YOLOv8s @ 320 / 640	153 / 584 ms
Jetson	YOLOv8s @ 320 / 480 / 640	29 / 31 / 33 ms
Laptop	YOLOv8s @ 320 / 640; YOLOv8m @ 640	9.0 / 9.9; 13.6 ms

JPEG of a 640 px frame: 87 KB at q90, 36 KB at q50; 26 KB and 11 KB at 320 px. Encoding takes at most 3 ms on the Pi.

Links on 3 October (all through one router)

LINK	ROUND TRIP	THROUGHPUT	85 KB FRAME	315 KB FRAME
Pi → laptop	4.9 ms	30 Mbit/s	25 ms	84 ms
Pi → Jetson	2.7 ms	13 Mbit/s	72 ms	224 ms
Jetson → laptop	4.0 ms	18.5 Mbit/s	43 ms	161 ms

Pi → Jetson is slowest because both are Wi-Fi clients of the router, so every packet crosses the air twice. On 5 October the same links measured about 5.6 Mbit/s.

PART 7

Every placement of the model

A placement chooses which device runs which layers and how data is encoded between them. `placement.py` evaluated all 1,150. Live runs then checked the important ones, with a 30 fps camera, 60 s runs, up to 4 repetitions, and detections returned to the camera.

PLACEMENT	FRAMES/S	ON TIME (≤300 MS)	LATENCY P50
Jetson alone (camera on the Jetson)	18.5	18.5	35 ms
Pi JPEG → laptop	15.2	14.7	107 ms
Pi JPEG → Jetson	10.9	10.1	150 ms
Jetson JPEG → laptop	9.0	8.1	139 ms
Pi split k1 → laptop (best split)	7.0	2.5	324 ms
Pi split k4 → laptop	4.2	0.0	419 ms
Pi split k4 → Jetson (PDF's "mothership")	2.7	0.0	691 ms
Pi JPEG → Jetson relay → laptop	3.4	0.5	570 ms
Pi k4 → Jetson k9 → laptop (two splits)	2.1	0.0	1,413 ms
Pi alone	1.6	0.0	589 ms

The model predicts two-device placements within about 30 %. It is too optimistic for three devices, because both Wi-Fi hops share one channel.

Encoding twice costs nothing

HOP 1	HOP 2	MAP CHANGE (POINTS, 95 % CI)
JPEG q90	INT8 at k4, k9 or k15	−0.32 to −0.35
INT8 at k1	INT8 at k2 (6 pairs)	−0.04 to +0.01
INT8 at k1	FP16 at k2 (3 pairs)	−0.03 to −0.01

Status: **VALIDATED** . Full write-up: `docs/hardware_results.md` .

PART 8

Corrections to Hardware Testing.pdf

PDF CLAIM	MEASURED
Pi YOLOv8s 786 ms	576 ms cool, 758 ms throttled, 950 ms on 1 thread
JPEG 359 KiB per frame	84–87 KB at q90
Drone-to-drone link "measured" 59.66 Mbit/s	That is our simulator's value; the real Pi → Jetson link was 13 Mbit/s
JPEG → cloud 38.8 ms at 57.8 frames/s over 17 Mbit/s	Not possible at that rate; measured 107 ms and 15 frames/s
Pi k4 → Jetson mothership as the best design	2.7 frames/s, never on time; JPEG to the Jetson is 4× better
Double INT8 loses 3.81 dB, so FP16 is needed	No accuracy loss (−0.04 to +0.01 points)
Jetson 41.3 ms	Correct for FP32; FP16 is 22.7 ms
Adaptive mission: 96.8 % compliance, +42.6 % endurance	A scripted scenario whose onboard power assumes 10 frames/s on a 1.3 frames/s device

PART 9

Adaptive controller v1

Design. The Pi chooses one option per frame: onboard YOLOv8s, JPEG q90 or q50 to the laptop or the Jetson, or the k1 split. It scores each option by predicted frame rate × probability of meeting the deadline × relative accuracy, minus an energy cost. Link speed is estimated from every frame. Probes test unused links. A stalled link is closed and reconnected, and switches use hysteresis.

Test. A scripted six-phase mission, 30 s per phase: good link, ground link congested, ground blackout, both links degraded, total blackout, recovery at low battery. 7 policies × 3 repetitions.

POLICY	ON-TIME FRAMES/S OVER THE MISSION
Adaptive v1	13.85
Best fixed (JPEG q50 → Jetson)	12.79
JPEG q50 → laptop	7.71
Hindsight best fixed per phase	14.43

Result: 96 % of the hindsight optimum and +8 % over the best fixed strategy. It won 2 of 3 repetitions; in the noisiest one it switched too often (56 times). The gain was modest because v1 could only pick one

the noisiest one it switched too often (56 times). The gain was modest because v1 could only pick one option at a time.

PART 10

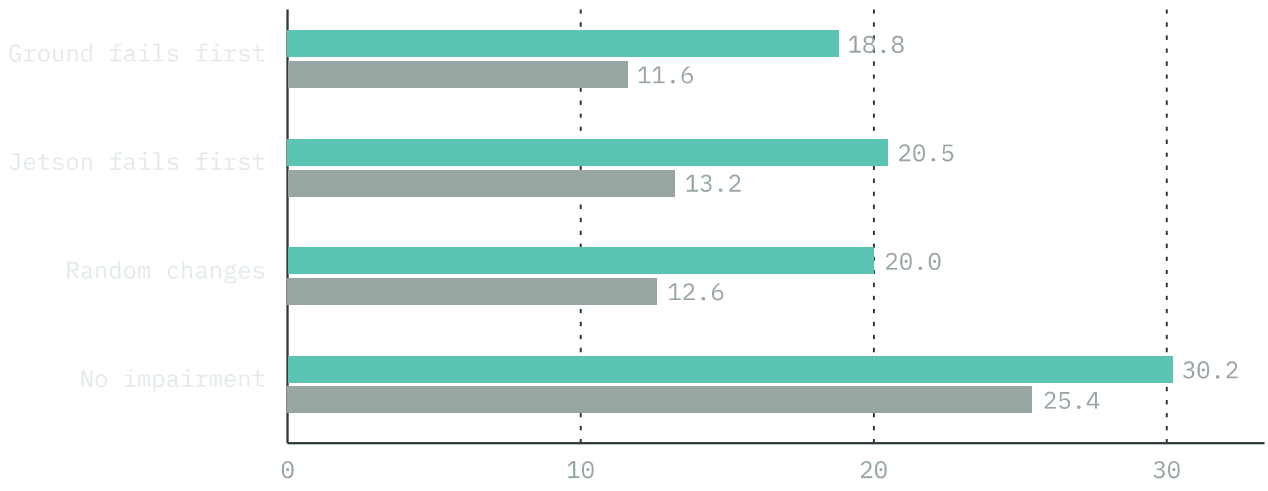
Adaptive controller v2: multi-server scheduling

What changed. The Pi, the laptop path and the Jetson path become three servers working in parallel, each holding at most one frame. For every camera frame the scheduler hands the newest frame to the idle server whose best variant adds the most accuracy-weighted on-time results per second. The variants are:

- **Onboard:** YOLOv8n@320, YOLOv8s@320, YOLOv8s@640.
- **Each path:** JPEG q90 or q50 at 640, 480 or 320 px for YOLOv8s.
- **Laptop only:** YOLOv8m@640.

Accuracy per variant comes from 5,000 COCO images, and timings are learned online.

Test. 4 scenarios (ground fails first, Jetson fails first, random link changes, no impairment) × 11 policies × 2 repetitions = 88 runs, in shuffled order.



Accuracy-weighted on-time detections per second, median of 2 repetitions. Green: adaptive v2 (A_full). Grey: best fixed strategy in that scenario.

SCENARIO	A_FULL	ALL SERVERS, NO ADAPTATION	PATHS ONLY	ONE SERVER	BEST FIXED	HINDSI
Ground fails first	18.8	16.9	16.4	13.8	11.6	
Jetson fails first	20.5	17.0	18.5	16.0	13.2	

Random changes	20.0	18.0	18.2	14.9	12.6
No impairment	30.2	24.6	19.7	20.2	25.4

- **Paired comparisons:** A_full was best in all 8 scenario × repetition pairs.
- **Availability:** 0.999 everywhere.
- **Pi power:** 0.7–3.1 W, against 2.4–3.1 W without adaptation.
- **Where the gain comes from:** parallel servers give most of the raw on-time count. Adaptation adds 10–23 % in accuracy-weighted terms and saves power.
- **Limits:** only 2 repetitions, and the scoring uses COCO accuracy weights, a proxy.

Status: **VALIDATED** on the proxy metric. Write-up: `docs/adaptive_results.md` .

PART 11

Literature check and novelty (re-checked 7 Oct 2026)

The check was redone after the forecasting, controller and swarm work, searching specifically for prior work on each new piece. Several ideas we use turn out to be **established**. The novelty is narrower than "forecasting" or "coordination" in general: it lies in specific measurements, in one coupling of forecaster and scheduler, and in the real-hardware evaluation. Rows need checking against the full papers before submission.

Prior work that already covers parts of what we do

PRIOR WORK	WHAT IT ALREADY DOES	CONSEQUENCE FOR US
Elf (MobiCom 2021)	Offloads parts of one frame to several edge servers in parallel	Parallel offloading is not new
Odessa (MobiSys 2011)	Adaptively decides offloading and parallelism across device and server for interactive perception	Adaptive device-plus-server use is not new
Chameleon, AWStream	Adapt resolution, frame rate and model to bandwidth on one path	Per-frame configuration adaptation is not new
Glimpse (SenSys 2015)	Tracks objects on the phone from stale server results to hide offloading delay	**Compensating offloading delay by tracking is not new**

Liu et al., edge-assisted detection for mobile AR (MobiCom 2019)	Offloads detection to the edge and tracks with motion vectors on the device to cover the delay	Same: latency-compensating tracking for offloaded detection is established
Streaming perception (ECCV 2020); PVT / PVT++ (ICCV 2023)	Define streaming AP; forecasting (Kalman, learned) to counter latency, including latency-aware tracking on UAVs	The metric and latency forecasting on drones are not new
BoT-SORT (2022) and similar trackers	Camera-motion compensation by sparse optical flow and a RANSAC affine fit between frames	**Our camera-motion estimation technique is standard**
JCAB (INFOCOM 2020, IEEE/ACM ToN 2022)	Jointly adapts each camera's settings and splits a shared uplink among several cameras	Coordinating cameras that share a link is not new
Drone edge analytics (e.g. SEC 2018); multi-UAV offloading studies	Onboard filtering; offloading and scheduling for several UAVs, mostly in simulation	Multi-UAV offloading is an active area
Split computing (Neurosurgeon and others)	Chooses a split point between device and server	We test this approach and find it loses

What we can claim as new

1. A measured negative result for split computing on drone hardware

None of the 1,150 placements of YOLOv8s over a Pi 5, a Jetson Orin Nano and a ground GPU, nor the live runs, beats sending a JPEG. The professor's serial sense → edge → cloud chain is the worst networked design. Backed by a closed-form dominance result that holds for any deadline, and by 19 detectors. `STRONG`

2. Measurements of how fast aerial detections go stale, with and without correction

On VisDrone video a detection keeps 41 % of its accuracy at 100 ms and 7 % at 267 ms. With a velocity forecaster: 77 % and 47 %. With camera-motion alignment: 85 % and 60 %. We found no prior measurement of this decay for aerial detection. `STRONG` (measurement)

3. Matching the offloading scheduler's objective to the forecaster in use

Our scheduler values each candidate result by the accuracy it will have *after* forecasting, using the measured post-forecasting decay curve. We show the choice of best policy changes with the forecaster: under strong forecasting a fixed big model won the clean link until the controller was rebuilt. The rebuilt controller then significantly beats v2 (+8.9 % median over 20 paired runs) and keeps a fresh detection 99.5–99.8 % of the time. Prior systems (Glimpse, Liu et al.) track to hide

delay but do not feed the forecaster's effect back into which server and setting to choose.
`MOST NOVEL` (verify against more recent offloading work)

4. A real three-tier drone testbed evaluated with streaming accuracy

Whole-frame scheduling across a drone CPU, an airborne-class edge GPU and a ground GPU, with the forecaster running live on the Pi. Live and offline scores are identical. Evaluated with task-level streaming AP on labeled aerial video, controlled impairments, 3–5 repetitions, paired tests, and controller inputs measured on data disjoint from the test videos. The method is a combination of known parts; the contribution is the measured evidence. `STRONG` (evaluation)

5. Hardware facts that matter for practice

Running the forecaster live slows the Pi's onboard inference (YOLOv8s-320: 170 → 412 ms) because they share the CPU. After a blackout the real link takes about 1.2 s to deliver again and stays slow (389 ms against 71 ms). A drone-scale simulator rebuilt from 44 real runs matches the hardware with r = 0.91 except in that recovery. `USEFUL` (measurement)

6. Swarm admission with radio link health

A coordinator that decides how many drones offload at once, rotates turns, and uses each radio's own link health to tell a drone's own outage apart from congestion. In calibrated simulation it removes collapse and starvation and keeps every loss at or below 1.9 %, on held-out seeds too. On real hardware with two cameras it rarely acts. Shared-link coordination itself is known (JCAB). The specific rule and the hardware calibration are ours, but the gains are simulation-only so far.
`MODERATE` (needs the 3–5-camera test)

7. (Added 7 Oct) Energy pricing that raises accuracy, measured on drone hardware

On a fanless drone computer, pricing energy against a live-measured power budget cut power by about 25 % and throttling-region time from 76 % to 17 %, and *increased* streaming accuracy significantly (+13 % median), because low-value onboard work also costs heat and CPU. Energy-aware offloading as such is a crowded field, so the claim is the measured three-way coupling and the result, not energy awareness itself. `STRONG` (needs a literature check before submission)

Claims we must not make

- That parallel offloading, adaptive device-plus-server use, or per-frame configuration adaptation is new (Elf, Odessa, Chameleon, AWStream).
- That forecasting or tracking to compensate offloading delay is new (Glimpse, Liu et al. 2019, streaming perception, PVT++).
- That camera-motion compensation by optical flow and RANSAC is our technique (BoT-SORT and

others).

- That coordinating cameras on a shared link is new (JCAB).
- That the evaluation is a flight test: links are lab Wi-Fi with injected impairments; the mesh campaign is pending.

Full notes: `docs/novelty_and_related_work.md` .

PART 12

Phase D: task-level test on drone video

Why it is needed

The v2 result scores each on-time result by its model's COCO accuracy. A reviewer will ask how accurate the drone's view really is. Phase D measures that directly on labeled drone video.

Data

- **Test set:** VisDrone2019-VID validation, 7 videos, 2,846 frames, about 108,000 labeled objects. Resized to 640 px wide; classes mapped to COCO exactly as in the paper.
- **Controller inputs, kept off the test set:** aerial accuracy per variant from VisDrone-DET val (548 images), and staleness decay from 8 VisDrone-VID training videos chosen by a fixed rule.

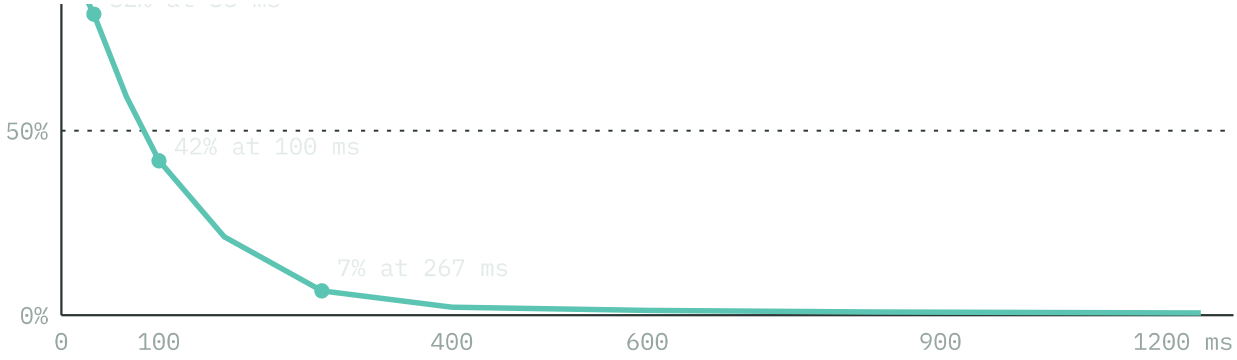
Aerial accuracy per variant

VARIANT	COCO MAP	AERIAL MAP
YOLOv8m 640 raw / q90 / q50	0.498 / 0.494 / 0.450	0.146 / 0.138 / 0.098
YOLOv8s 640 raw / q90 / q50	0.444 / 0.441 / 0.409	0.107 / 0.105 / 0.083
YOLOv8s 480 q90 / q50	0.413 / 0.380	0.073 / 0.058
YOLOv8s 320 raw / q90 / q50	0.357 / 0.350 / 0.317	0.040 / 0.039 / 0.028
YOLOv8n 640 / 320 raw	0.368 / 0.281	0.072 / 0.026

On aerial images 320 px loses 63 % of accuracy, against 20 % on COCO, because objects are tiny.

Staleness decay





AP of detections from an older frame, scored on the current frame, relative to fresh detections (YOLOv8s 640, VisDrone-VID train).

Controller v3 (A_stream)

For each idle server and variant, v3 predicts the result's latency L and how long it will stay the drone's newest result, T. Its value is the variant's aerial accuracy times the average freshness over that window, times the chance it arrives within 1 s. The drone keeps whichever delivered result currently has the highest accuracy × freshness. For a single server, this rule maximizes the long-run average quality exactly; with several servers it is a greedy approximation. Model write-up: `docs/controller_model.md` .

Verification of the evaluation

- The streaming-AP scorer reproduces the official ultralytics validator: 0.0967 against 0.0976 mAP, with the residual from FP16.
- The fast NMS gives the same AP as ultralytics' (0.0845 vs 0.0844).

Campaign design

- **Scenarios:** 4 (as in v2).
- **Policies:** 9.
  - Three controller versions as ablations: v3, v2 with aerial accuracy, and v2 with COCO accuracy.
  - Three literature-style baselines: all servers with no adaptation, single-path bitrate adaptation, and rule-based fallback.
  - Three fixed strategies: laptop YOLOv8m q90, Jetson YOLOv8s q90, Pi YOLOv8n 320.
- **Repetitions:** 5. That makes 180 runs of about 3 minutes, in shuffled order, with 95 % confidence intervals and paired tests.

Campaign history

ATTEMPT	WHAT HAPPENED	OUTCOME
1	A network outage, then the Jetson's clock lock was found cleared by a	INVALIDATED 6

	reboot (GPU at 306 MHz)	runs
2	NMS at the low confidence needed for AP took up to 130 ms on the Jetson, hurting remote servers unevenly	**INVALIDATED** 59 runs
3	The controller stopped using a server after its slow warm-up frames, so the Jetson and YOLOv8m went unused	**INVALIDATED** 50 runs
4	All fixes in; the smoke test shows the Jetson used throughout and availability 0.91–0.99. Paused for congested Wi-Fi	**PENDING** 3 of 180 runs
5	Real-world version on the current shared Wi-Fi (Pi to laptop 22 Mbit/s, Pi to Jetson 18.5 Mbit/s, Pi signal −79 dBm): same code, 3 repetitions. No outages or errors	**VALIDATED** 108 of 108 runs

What the invalid runs showed

In attempt 3 (1–2 repetitions per cell), the adaptive controllers were not ahead. In the ground-fails-first scenario, "all servers, no adaptation" scored 0.021 sAP against 0.015 for v3. On a clean network, fixed laptop YOLOv8m scored 0.044 against 0.035. Part of this was the warm-up bug, which kept v3 off the Jetson and off YOLOv8m. Whether v3 wins once it is fixed is the open question this campaign will answer.

Answer (attempt 5 below and Part 14): the original v3 did not win. Rebuilt with forecasting it matches v2 on accuracy and clearly beats it on availability.

Results on the shared Wi-Fi (attempt 5, 108 runs)

Streaming AP (mAP@[.5:.95] of what the drone holds at every frame, scored on VisDrone-VID val), mean ± 95 % interval over 3 repetitions, with the quality-aware holding rule. The last column applies the motion forecaster of Part 14.

POLICY	GROUND FAILS FIRST	JETSON FAILS FIRST	RANDOM	NO IMPAIRMENT	MEDIAN
v2 multi-server (A_full)	**0.0207 ± 0.0018**	0.0168 ± 0.0252	**0.0201 ± 0.0017**	0.0349 ± 0.0092	
All servers, no adaptation	0.0204 ± 0.0041	0.0211 ± 0.0027	0.0186 ± 0.0082	0.0311 ± 0.0031	
v2 with aerial accuracy	0.0185 ± 0.0037	0.0202 ± 0.0064	0.0180 ± 0.0040	0.0246 ± 0.0253	

v3 staleness-aware (A_stream)	0.0160 ± 0.0027	0.0211 ± 0.0042	0.0170 ± 0.0052	0.0216 ± 0.0332
Single-path rate adaptation (laptop)	0.0127 ± 0.0039	**0.0216 ± 0.0063**	0.0173 ± 0.0031	0.0309 ± 0.0099
Fixed laptop YOLOv8m q90	0.0096 ± 0.0032	0.0195 ± 0.0077	0.0121 ± 0.0053	**0.0356 ± 0.0069**
Rule-based fallback	0.0091 ± 0.0025	0.0156 ± 0.0045	0.0095 ± 0.0044	0.0299 ± 0.0040
Fixed Jetson YOLOv8s q90	0.0169 ± 0.0048	0.0070 ± 0.0012	0.0074 ± 0.0051	0.0220 ± 0.0081
Fixed onboard YOLOv8n 320	0.0073 ± 0.0005	0.0071 ± 0.0013	0.0069 ± 0.0013	0.0070 ± 0.0003

Paired comparison: v2 (A_full) against each policy

AGAINST	MEAN DIFFERENCE ± 95 %	V2 AHEAD	MEDIAN RELATIVE GAIN
Fixed onboard YOLOv8n 320	+0.0160 ± 0.0048	11 / 12	+201 %
Rule-based fallback	+0.0071 ± 0.0039	10 / 12	+75 %
Fixed Jetson YOLOv8s q90	+0.0098 ± 0.0034	11 / 12	+68 %
Fixed laptop YOLOv8m q90	+0.0039 ± 0.0047	9 / 12	+37 %
v3 staleness-aware	+0.0042 ± 0.0049	10 / 12	+16 %
v2 with aerial accuracy	+0.0028 ± 0.0045	11 / 12	+16 %
Single-path rate adaptation	+0.0025 ± 0.0038	10 / 12	+14 %
All servers, no adaptation	+0.0003 ± 0.0030	8 / 12	+4.5 %

- **v2 is the best policy overall.** It is ahead of every other policy on average and first or within the interval of first in every scenario except "Jetson fails first", where one bad repetition widened its interval (± 0.025).
- **It is statistically significant against three baselines** (onboard, Jetson, rule-based fallback). Against the other five the 95 % interval still crosses zero, because 3 repetitions on a shared Wi-Fi give wide intervals. The mesh campaign with 5 repetitions is meant to settle these.
- **On a clean link, a fixed strong server is as good.** Fixed laptop YOLOv8m (0.0356) and v2 (0.0349) are tied. Adaptation pays off when links fail.
- **v3 did not deliver.** Its staleness objective, built from the no-forecasting decay curve, favours fast, low-accuracy results too strongly. Its availability was 0.77–0.90.
- **Availability:** all servers without adaptation keeps 0.999 everywhere, but at 2.3–3.1 W of Pi power. v2 reaches 0.77–0.98 at 0.9–2.7 W.

Files: `experiments/results/hardware/vid_campaign_wifi/` (runs and the evaluation cache), `vid_campaign_wifi_summary.json/.csv`, logs `vid_campaign_wifi_summary.log` (paired against v3) and `vid_campaign_wifi_summary_refA_full.log` (paired against v2).

PART 13

Phase E: swarm simulation calibrated to the hardware

Why it is needed

The testbed has one camera drone. A swarm means several drones sharing one network, and the question is how each strategy behaves as the swarm grows. Buying ten Pis is not needed for that: the simulator from Phase B (checked against ns-3, Simu5G and OMNeT++) was rerun with the numbers measured on the hardware instead of the paper's assumed ones.

Calibration

INPUT	VALUE	SOURCE
Camera drone, Pi 5	758 ms per YOLOv8s frame	Measured, sustained and throttled (Part 6)
Camera drone, Jetson-class	35 ms per frame	Measured Jetson Orin Nano, FP16 with pre- and post-processing
Shared network, Wi-Fi access point	25 Mbit/s	Measured goodput of the testbed router (22–30 Mbit/s)
Shared network, cellular cell	100 Mbit/s	The paper's cell

How the capacity is divided	Water-filling (equal shares) or capped	Capped reproduces the Simu5G result that a real scheduler serves about 7 drones and starves the rest (check O2)
Payload sizes, server times, deadline	Measured; 300 ms deadline; 30 fps camera	As in the paper

Grid: 2 camera drones × 2 networks × 2 sharing rules × 2 channel profiles (moderate fading, blackouts) × 6 swarm sizes (1, 2, 3, 5, 10, 20) × 4 strategies (onboard, split at layer 4 with INT8+zstd, JPEG q90, adaptive) = 384 cells, each 5 seeds of 120 s. Metric: on-time frames per second per drone. Script `validation/hardware/swarm_calibrated.py` ; it ran on the laptop CPU in 15 minutes. **VALIDATED** as a simulation; it is not a swarm measurement.

Pi camera drones, moderate channel, capped sharing

DRONES	ONBOARD	SPLIT K4	JPEG Q90, WI-FI	ADAPTIVE, WI-FI	JPEG Q90, CELL	ADAPTIVE, CEL
1	0	0	14.8	15.0	14.8	15
2	0	0	14.8	15.0	14.8	15
3	0	0	11.1	15.0	14.8	15
5	0	0	5.5	15.0	14.8	15
10	0	0	0	8.3	11.2	14
20	0	0	0	4.2	5.2	12

Jetson-class camera drones, moderate channel, Wi-Fi 25 Mbit/s

DRONES	ONBOARD	SPLIT K4	JPEG Q90	ADAPTIVE, EQUAL SHARES	ADAPTIVE, CAPPED
1	15.0	7.7	25.8	29.6	29.6
2	15.0	4.5	16.5	29.6	29.6
3	15.0	0	11.1	20.1	20.1
5	15.0	0	6.7	18.3	18.3
10	15.0	0	0 / 3.3	16.5	16.8
20	15.0	0	0 / 1.7	14.0	14.9

JPEG q90 at 10 and 20 drones: equal shares / capped. On the 100 Mbit/s cell the Jetson-class adaptive drone keeps 29.5 frames/s up to 10 drones and 18.9–19.2 at 20.

What it shows

A Pi swarm must offload

At 758 ms per frame, the Pi alone, and the Pi running the first layers of a split, never meet the 300 ms deadline at any swarm size. Every useful frame comes from sending a JPEG. The simulated Pi prepares at most every other camera frame, so 15 frames/s is its ceiling.

Fixed JPEG collapses as the swarm grows; adaptive degrades gracefully

On the shared 25 Mbit/s Wi-Fi, fixed JPEG q90 falls from 14.8 frames/s per drone (1–2 drones) to 5.5 at 5 drones and 0 at 10. The adaptive controller holds 15.0 up to 5 drones (2.7× fixed JPEG) by moving to JPEG q50, at a 7 % accuracy cost (mAP 0.409 against 0.440), and still delivers 8.3 at 10 drones.

With a GPU on board, adaptive doubles the onboard rate in small swarms

A Jetson-class drone alone gives 15 frames/s (35 ms is just over one camera period). Offloading the rest gives 29.6 at 1–2 drones and 18.3 at 5.

Splitting is worse than JPEG in every cell

The split at layer 4 never beats JPEG q90, at any size, on either network. This matches the paper and the hardware placements.

Adaptive is best or tied in 78 of 95 comparisons

Here a comparison is one setting at one swarm size where some strategy delivers frames; each pits adaptive against the best fixed strategy for that setting.

- In 12 of the 17 losses it is behind by at most 0.5 %. These are small Pi swarms during blackouts, where it occasionally tries a lower JPEG quality.
- The other 5 losses are Jetson-class swarms of 10–20 drones on Wi-Fi. There, staying onboard is best and adaptive is 0.8–8.3 % behind, because it keeps probing a network that is too crowded to help.

The limit is coordination, not the per-drone controller

Each drone decides alone and cannot see the others' load. With 20 Pi drones on Wi-Fi and equal

shares, every drone gets about 1.25 Mbit/s, which is too little for any frame to arrive on time, and output falls to 0. Under capped sharing the mean is 4.2 frames/s, but some drones get 0 (the minimum per drone is 0 at 10 and 20 drones). A swarm-level admission rule (only as many drones offload as the network can carry, the rest degrade) should fix both; it is tested below.

Swarm coordination (added)

A coordinator on the ground side knows the shared capacity and each drone's reported link estimate (a few bytes per drone per second). Once a second it picks how many drones K may offload at the same time, the K that maximizes the swarm's expected on-time detections. For each drone it lists the offload settings that still fit when K drones use them at their predicted frame rate. Admitted drones keep choosing frame by frame among those settings and onboard. The others run onboard if that is useful, and otherwise skip the frame. The admitted set rotates round-robin for fairness. When there is no contention, coordination switches off and every drone behaves exactly as before. Code: `coordination="admission"` in `simulation/stream_sim.py`.

The one setting, the capacity multiple the coordinator plans with (1.1), was chosen on separate tuning seeds (100–101). The design went through four versions checked on the study seeds 0–4, so the final version was also run on fresh seeds 5–9.

CASE (MODERATE CHANNEL)	UNCOORDINATED	COORDINATED	CHANGE
20 Pi drones, Wi-Fi, equal shares	0.0 (worst drone 0.0)	4.15 (worst 4.08)	collapse removed
10 Pi drones, Wi-Fi, capped router	8.32 (worst drone 0.0)	8.29 (worst 7.65)	no starvation
20 Pi drones, Wi-Fi, capped router	4.16 (worst drone 0.0)	4.12 (worst 3.62)	no starvation
5 Jetson-class drones, Wi-Fi	18.3	22.4	+22 %
10 Jetson-class drones, Wi-Fi	16.5	18.7	+13 %
20 Jetson-class drones, Wi-Fi	14.0 (below onboard 15)	16.9 (above onboard)	+21 %
20 Jetson-class drones, cell, equal shares	18.9	22.6	+20 %
20 Jetson-class drones, cell, capped router	19.2	17.8	−7.4 %

Better in 16, tied in 74, worse in 6 of 96 cases, and identical on held-out seeds

Seeds 0–4 and fresh seeds 5–9 give the same split. Five of the six losses are within 1.7 %.

`VALIDATED` (simulation)

The one real loss is a scheduler the coordinator cannot see

The one real loss is a scheduler the coordinator cannot see

On the cell with the capped router (serves 7 drones at a time), admitting 10 drones starves 3 of them. Learning K from each drone's achieved throughput closes this gap, but costs elsewhere (ablation below).

One case it does not catch

3 Jetson-class drones on Wi-Fi stay at 20.1, because every drone's own choice looks admissible. An earlier version that locked settings reached 27, but cost 2–8 % in small swarms.

Ablation: a coordinator that learns from feedback

To close the two gaps above, a feedback layer was tried (`coord_feedback` in `stream_sim.py`), evaluated on the tuning seeds 100–101 over 80 cases:

VARIANT	MEAN GAIN OVER UNCOORDINATED	BETTER / WORSE	CAPPED-ROUTER CASE (20 JETSON, CEL
Coordinator (main result)	+16.0 %	16 / 6	−7.6
+ cap K at the number of drones actually served	+15.7 %	16 / 6	−7.1
+ that, and shared lateness cuts planned capacity (AIMD)	+15.7 %	19 / 6	+0.7
First AIMD version (any late drone counts)	+15.0 %	19 / 21	+3.7

Feedback fixes the capped-router loss, but it mistakes blackouts on individual links for contention and costs Pi swarms in blackouts. Overall it does not beat the plain coordinator, so the plain coordinator stays the main result and feedback is reported as a negative ablation. The 3-drone case never shows late frames, only a lower frame rate, so lateness cannot detect it. Telling an own-link blackout apart from

late frames, only a lower frame rate, so lateness cannot detect it. Telling an own-link blackout apart from contention (for example from the radio's signal level) is the open problem. **NOT SUPPORTED** (feedback from goodput estimates)

Coordinator with link health from the radio (new main result)

The fix. A modem knows its own link quality instantly from its signal level, while the coordinator above only had a smoothed goodput estimate. In this version each drone also reports its radio's own link capacity over the last window, its worst 10 ms slot (`coord_feedback="rssi"`). A drone whose own link was down is then excluded from both feedback rules. Its late frames no longer count as congestion, and its empty window no longer counts as the router starving it. On the tuning seeds, the first version that applied this only to lateness did not help (+15.7 %, worst −7.6 %). Applying it to the starvation rule as well did.

SEEDS	COORDINATOR	BETTER / TIED / WORSE (96 CASES)	WORST LOSS	CAPPED-ROUTER CASE (20 JE
Study 0–4	Plain	16 / 74 / 6	−7.4 %	17.78 (uncoord
Study 0–4	**With radio link health**	**18 / 72 / 6**	**−1.9 %**	
Held-out 5–9	Plain	16 / 74 / 6	−7.3 %	17.76 (uncoord
Held-out 5–9	**With radio link health**	**18 / 73 / 5**	**−1.7 %**	

It keeps every large gain unchanged (20 Pi drones 0 → 4.15; Jetson-class +22 % at 5 and +21 % at 20 on Wi-Fi; +20 % at 20 on the cell) and removes the one real loss. All remaining losses are at most 1.9 %. Same result on held-out seeds. **VALIDATED** (simulation)

Two camera drones on real hardware

Setup. The Jetson ran as a second camera drone: the same controller, onboard YOLOv8 on its GPU (FP16; YOLOv8s-640 in 39 ms), offloading to the laptop. It also kept serving the Pi's edge path, so its GPU was shared. Both cameras streamed the same labeled video and sent to the one laptop, which ran one worker per camera on its single GPU over its single Wi-Fi link, plus the live coordinator. The coordinator connects out to both cameras, takes a report from each every second and decides which cameras may use the laptop (`coordinator.py` , `tools/arun2.sh`). Every camera ran forecaster v2 live.

Campaign. 2 scenarios (clean link; failure mission) × 4 settings × 3 repetitions = 24 runs, shuffled, 21:48–22:58 on 6 October, no outages. Settings:

- **Pi alone:** the reference;
- **two cameras, v2, coordinator logging only;**
- **two cameras, v2, coordinator active;**
- **two cameras, fixed JPEG to the laptop's YOLOv8m.**

SCENARIO	SETTING	PI CAMERA SAP	JETSON CAMERA SAP	SWARM MEAN	PI AVAILABILITY
Clean link	Pi alone	0.0806 ± 0.0164	—	—	0.987
Clean link	Two cameras, v2	0.0715 ± 0.0184	0.0782 ± 0.0025	0.0749 ± 0.0082	0.988
Clean link	Two cameras, v2 + coordinator	0.0718 ± 0.0167	0.0786 ± 0.0016	0.0752 ± 0.0078	0.979
Clean link	Two cameras, fixed laptop	0.0772 ± 0.0165	0.0631 ± 0.0060	0.0702 ± 0.0102	0.962
Failure mission	Pi alone	0.0443 ± 0.0072	—	—	0.757
Failure mission	Two cameras, v2	0.0360 ± 0.0035	0.0784 ± 0.0007	0.0572 ± 0.0018	0.682
Failure mission	Two cameras, v2 + coordinator	0.0394 ± 0.0028	0.0787 ± 0.0013	0.0590 ± 0.0015	0.695
Failure mission	Two cameras, fixed laptop	0.0215 ± 0.0019	0.0663 ± 0.0303	0.0439 ± 0.0152	0.190

Streaming AP with forecaster v2 (live camera motion), mean ± 95 % interval over 3 repetitions. The Jetson camera's link was not impaired in the mission (netem acts on each camera's own interface for the paths it has).

Sharing the ground station has a real cost

The Pi camera loses 0.0088 ± 0.0021 sAP when a second camera shares the laptop (paired, 6 of 6): −11 % on a clean link, −19 % in the failure mission. `VALIDATED` (hardware, 2 cameras)

Adaptive control beats fixed offloading in a swarm too

Swarm mean sAP +0.0090 ± 0.0063 (v2) and +0.0101 ± 0.0074 (v2 + coordinator) over two

swarm-mean SAP +0.0090 ± 0.0062 (v2) and +0.0101 ± 0.0074 (v2 + coordinator) over two cameras both sending to the laptop, ahead in 6 of 6 pairs each. The fixed Pi camera's availability collapses to 0.19 in the mission. **VALIDATED** (hardware, 2 cameras)

With two cameras the coordinator rarely acts

It admitted both cameras in about 124 of 125 one-second windows: by its own rule, two cameras sharing the laptop are worth more than taking turns. Coordinated and uncoordinated runs therefore differ only by noise (+0.0011 ± 0.0027 swarm mean, not significant). The live mechanism works (reports every second, decisions logged, no failures). Its benefit is expected from 3–5 cameras on, as in simulation, which needs the extra boards. **INCONCLUSIVE** (needs more cameras)

Files: `experiments/results/hardware/swarm_duo/` (per-camera runs, detections, live forecaster data, coordinator logs `coord_*.json`), `swarm_duo_summary.json`, log `swarm_duo.log`.

Does the simulator match the hardware?

To make the swarm simulation credible, the simulator was checked against the real runs (`validation/hardware/sim_vs_real.py`). Every live run of a fixed single-server strategy was rebuilt in the simulator from that run's own measurements, with nothing fitted:

- link speed in unimpaired phases: the median the camera measured in that run;
- rate-limited phases: the configured limit, capped by that speed;
- blackouts: zero;
- frame size, encoding time, server time and round trip: measured.

The simulator then predicted on-time results per second for each of 528 phases (44 runs across the three Wi-Fi campaigns), compared with the measured values.

PHASE TYPE	PHASES	MEASURED	PREDICTED (PIPELINED)	PREDICTED (STOP-AND-WAIT)	ERROR, STO
Unimpaired	191	14.2	about 2×	15.2	3.7
Rate-limited	211	2.6	—	3.6	1.4
Blackout	85	0.0	0.0	0.0	
Unimpaired, right after a blackout	41	3.4	—	14.6	
All except post-blackout	**487**	**6.7**	—	**7.5**	**2.0**

- **Protocol matters.** The simulator first predicted about twice the measured rate on clean links. The cause was not the network model but the controller's protocol. On the hardware, a path gets its next frame only after the previous result has returned (one frame in flight), while the simulator pipelined. With that rule modelled through the simulator's existing link-hold parameter (server time + round trip), it matches with correlation 0.91 and an average error of 2.0 results/s.
- **The one thing it misses: recovery after a blackout.** On the hardware, results come back a median 1.2 s after a blackout ends (90th percentile 1.5 s). The link then stays slow for the rest of the phase: median latency 389 ms against 71 ms before the blackouts, so most results miss the 300 ms deadline. The simulator assumes instant full recovery and over-predicts these phases about 3.5×. The cause is not isolated yet; TCP congestion state after heavy loss is the main candidate. It makes the simulation optimistic for strategies that depend on one link after an outage, so it does not favour the multi-server controllers.

Files: `experiments/results/hardware/sim_vs_real.json` (pipelined), `sim_vs_real_stopwait.json` (with the protocol rule; all rows and the recovery times).

> **Limits of this simulation**
>
> The simulated adaptive controller is the paper's single-path one. It does not have the hardware controller's parallel multi-server dispatch, its smaller models, or its 480 and 320 px variants. Accuracy here is COCO mAP, not aerial accuracy, and the 300 ms deadline overstates quality on drone video (Part 12). Treat the numbers as trends with swarm size. The two-camera hardware test planned for the mesh is the real-world check.

PART 14

Phase F: motion forecasting of stale detections

Why it is needed

Part 12 measured that aerial detections lose most of their accuracy within a few frames: 58 % gone at 100 ms and 93 % at 267 ms, because the drone and the objects move. Every offloading strategy shows the drone results that are 50–300 ms old. Instead of only choosing fresher results, the drone can move each old box to where its object should be now.

Method

- When a result arrives, its boxes are matched to the previous result's boxes: same class, IoU at least 0.3, with the previous boxes first moved forward to the new capture time.
- Each matched box gets a velocity: 0.3 × (displacement / time) + 0.7 × its previous velocity

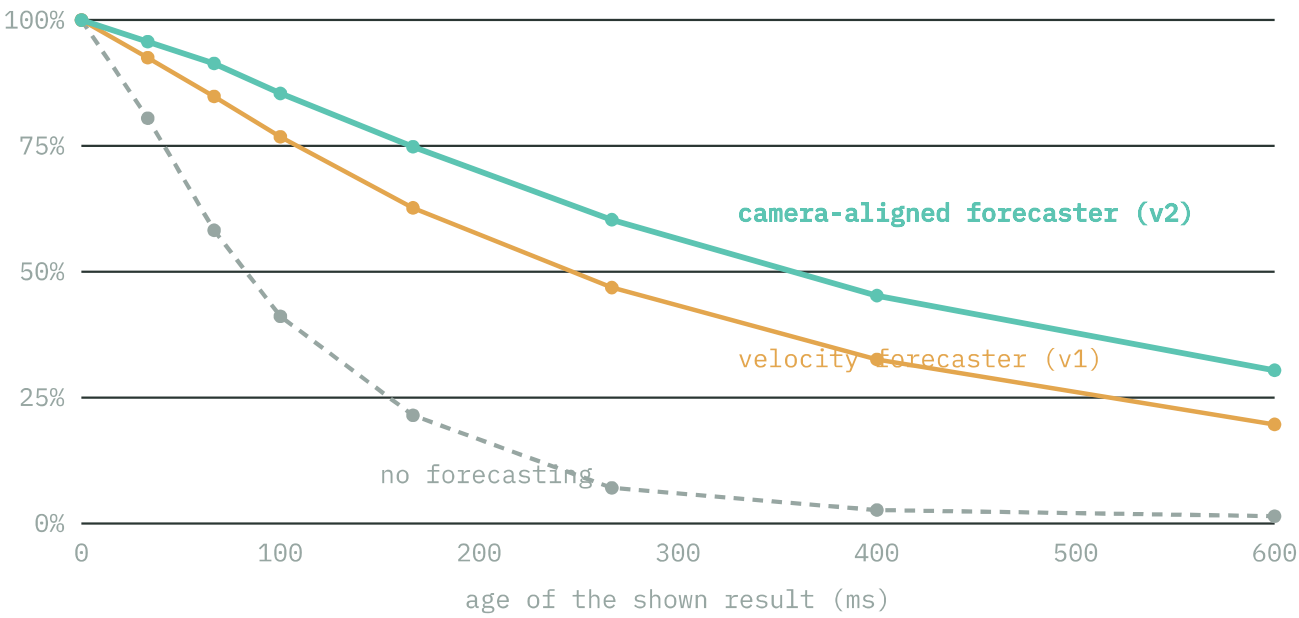
- Each matched box gets a velocity: 0.3 × (displacement / time) + 0.7 × its previous velocity. Unmatched boxes get zero velocity.
- When a result is shown, every box is shifted by velocity × its age (at most 1 s). The box size is kept.
- It is causal: it only uses results that have already arrived. It needs no extra traffic. On the Pi 5 it costs 3.9 ms per arriving result (100 boxes) and 0.06 ms per displayed frame (measured). Code: `validation/hardware/forecast.py`, `--forecast` in `stream_ap.py`.

Tuning, kept apart from the test

The five settings (smoothing, match threshold, minimum confidence, camera-motion fallback, horizon; 128 combinations) were chosen on VisDrone-VID **train**: 2,241 frames, YOLOv8s detections computed on the laptop CPU so the campaign's GPU was untouched. The replay used one result every 2 frames, shown 0–600 ms late. The chosen setting was then frozen and applied to the 108 campaign runs on VisDrone-VID **val**.

The decay curve below was then measured with the frozen forecaster: each result is shown exactly the stated time after its capture, with one result every 2 frames. The curve without forecasting reproduces the earlier GPU measurement of Part 12 (41 % against 42 % kept at 100 ms; 22 % against 21 % at 167 ms), which cross-checks the CPU detections.

AGE OF THE SHOWN RESULT	0 MS	100 MS	167 MS	267 MS	400 MS	600 MS
Accuracy kept, no forecasting	100 %	41 %	22 %	7 %	2.6 %	1.4 %
Accuracy kept, velocity forecaster (v1)	100 %	77 %	63 %	47 %	33 %	20 %
Accuracy kept, camera-aligned forecaster (v2)	100 %	**85 %**	**75 %**	**60 %**	**45 %**	**30 %**



Share of fresh accuracy kept when a detection is shown late, VisDrone-VID train, one result every 2 frames, each result shown exactly the stated time after capture. Both forecasters frozen before this measurement

(staleness_decay_forecast.json , staleness_decay_forecast_v2.json).

On the real campaign

On the 108 Wi-Fi runs it raised streaming AP for **every** policy: median +32 % (all servers, no adaptation), +37 % (v2), +48 % (v3), +47 % (fixed laptop), +53 % (rule-based fallback), +67 % (onboard). The ranking of the policies hardly changes. Forecasting stacks on top of any scheduler. **VALIDATED**

> **What it changes for the controller**
>
> v3's staleness objective was built on the no-forecasting decay curve, which drops to 7 % at 267 ms. With forecasting the real curve is far flatter (47 % at 267 ms). So v3 values freshness too much. v3 was therefore rebuilt on the forecasting curve and tested next.

Forecaster v2: measuring the camera's own motion

Idea. On a drone most of the apparent motion in the image is the camera's own: the drone flies, turns and climbs. The first forecaster could only guess it from the boxes. But the drone holds every camera frame, so it can **measure** it. For each new frame: about 200 corner points on the background, tracked from the previous frame with pyramidal Lucas–Kanade optical flow, and a RANSAC fit of a similarity transform (rotation, scale, shift). Moving objects are rejected as outliers. Chaining these per-frame steps gives the camera motion between any two frames (`validation/hardware/egomotion.py`).

Forecast. An old detection is first carried from its capture frame to the current frame with the measured camera motion, then moved by its object's own velocity. That velocity is estimated after removing the camera motion, so it describes only the object.

Tuning, again on train only. 24 new settings (smoothing, match threshold, with or without object velocity, with or without box-size extrapolation) were run alongside the v1 winner on the same training replay. The best uses camera alignment plus object velocity (smoothing 0.3, IoU 0.3). Size extrapolation slightly hurt, and camera alignment alone barely helps (AP 0.039 against 0.029 at 167 ms). It raised mean AP over the lags from 0.0755 (v1) to 0.0942 (+25 %). This setting was frozen before any test run was scored. The old forecaster reproduces its earlier numbers exactly in the new code (0.1002 and 0.0513 at 167 and 400 ms).

Cost on the drone. Camera tracking takes 2.7 ms per frame on the Pi 5 (p95 3.1 ms, measured on VisDrone frames), about 8 % of one core at 30 fps.

On all 144 real Wi-Fi runs (rescored offline with the frozen setting):

> **Better than the v1 forecaster in every one of the 144 runs**
>
> Over the 108-run campaign: median +51 %, smallest gain +25 %, 108 of 108 runs improved. Over the 36-run campaign: median +37 % to +45 % per policy, 36 of 36 improved. **VALIDATED**

the 36-run campaign. Median +87 % to +145 % per policy, 36 of 36 improved. VALIDATED

Roughly doubles streaming AP compared with no forecasting

Median gain per policy: +80 % (all servers, no adaptation) to +156 % (rule-based fallback). For v2 +85 %, for v3 + forecasting +92 %, for fixed laptop YOLOv8m +133 %. VALIDATED

Strong forecasting changes which policy wins

Streaming AP with the v2 forecaster, 108-run campaign (mean ± 95 % interval, best in each scenario in bold):

POLICY	GROUND FAILS FIRST	JETSON FAILS FIRST	RANDOM	NO IMPAIRMENT
v2 with aerial accuracy	**0.0391 ± 0.0026**	0.0444 ± 0.0148	**0.0402 ± 0.0123**	0.0534 ± 0.0433
v2 multi-server (A_full)	0.0385 ± 0.0052	0.0338 ± 0.0412	0.0360 ± 0.0112	0.0610 ± 0.0130
v3 (original)	0.0375 ± 0.0077	**0.0486 ± 0.0084**	0.0347 ± 0.0129	0.0474 ± 0.0618
Single-path rate adaptation	0.0252 ± 0.0080	0.0465 ± 0.0174	0.0359 ± 0.0015	0.0679 ± 0.0232
All servers, no adaptation	0.0368 ± 0.0057	0.0385 ± 0.0022	0.0343 ± 0.0149	0.0535 ± 0.0072
Fixed laptop YOLOv8m q90	0.0195 ± 0.0021	0.0476 ± 0.0064	0.0337 ± 0.0175	**0.0799 ± 0.0071**
Rule-based fallback	0.0246 ± 0.0013	0.0377 ± 0.0040	0.0273 ± 0.0122	0.0633 ± 0.0083
Fixed Jetson YOLOv8s q90	0.0378 ± 0.0020	0.0151 ± 0.0008	0.0249 ± 0.0239	0.0553 ± 0.0115
Fixed onboard YOLOv8n 320	0.0186 ± 0.0010	0.0183 ± 0.0028	0.0181 ± 0.0022	0.0178 ± 0.0009

- **When links fail, an adaptive controller is still best.** In all three impaired scenarios the top policy is adaptive (v2 with aerial accuracy twice, v3 once). Fixed strategies collapse when their one server fails: the laptop to 0.0195 when the ground link fails, the Jetson to 0.0151 when the Jetson fails.
- **On a clean link, the big model wins clearly.** Fixed laptop YOLOv8m reaches 0.0799 against 0.0610 for v2. With strong forecasting, a slower but more accurate result loses little to its age, so accuracy matters more than speed.
- **Why the controllers lag there.** All controllers were built for no forecasting or the v1 forecaster. Their objectives still over-value speed and under-use YOLOv8m. The fix follows directly from claim 4 in

Part 16: rebuild the objective on the v2 decay curve. That is the next hardware test.

- **Paired over all scenarios**, v2 still beats onboard (+119 %, significant) and the Jetson (+9 %, significant). Against the other policies, intervals cross zero.

Files: `experiments/results/hardware/forecast_tuning_v2.json` , `staleness_decay_forecast_v2.json` , `vid_campaign_wifi_summary_v2.json/.log` , `vid_campaign_wifi_fc_summary_v2.json/.log` ; camera-motion chains in `datasets/*/egomotion.npz` .

v3 rebuilt on the forecasting decay curve: paired Wi-Fi campaign

v3's objective was rebuilt on the forecasting decay curve above (`--decay staleness_decay_forecast.json` ; policy "v3 + forecasting"). It ran in a paired campaign against v2 and the old v3 on the same Wi-Fi: 4 scenarios × 3 repetitions × 3 policies = 36 runs in shuffled order, 06:52–08:29 on 6 October, no outages. Results are scored with forecasting applied. The quality-aware holding rule uses the forecasting curve, as the drone would if it forecasts.

SCENARIO	V2 SAP	OLD V3 SAP	V3 + FORECASTING SAP	V2 AVAILABILITY	OLD V3 A
Ground fails first	0.0223 ± 0.0116	0.0232 ± 0.0071	**0.0242 ± 0.0216**	0.83 ± 0.11	
Jetson fails first	0.0243 ± 0.0388	0.0327 ± 0.0203	**0.0349 ± 0.0089**	0.75 ± 0.37	
Random	0.0273 ± 0.0105	**0.0277 ± 0.0128**	0.0218 ± 0.0033	0.92 ± 0.17	
No impairment	**0.0618 ± 0.0081**	0.0616 ± 0.0051	0.0559 ± 0.0184	**0.998 ± 0.004**	

- **Accuracy: no significant difference.** Over the 12 pairs, v3 + forecasting is ahead of v2 on the mean by 0.0003 ± 0.0062 sAP, but v2 is ahead in 8 of 12 pairs (median +13.5 % for v2). v3 + forecasting wins both failure scenarios and loses the random and clean ones. **INCONCLUSIVE**
- **Availability: a clear, consistent gain.** v3 + forecasting held a detection under 300 ms old 99.0–99.9 % of the time in every scenario, with very tight intervals. v2 managed 75–92 % under impairment and the old v3 75–88 %. Mean age of information fell from 191–212 ms (v2) to 142–156 ms in the two failure scenarios. **VALIDATED** (Wi-Fi, 3 repetitions)
- **The earlier v2 lead over the old v3 did not repeat.** In campaign A, v2 led the old v3 by a median 16 %. In this session they are tied (median +0.2 %). Between-session variation on a shared Wi-Fi is as large as the differences between adaptive controllers. Only the controlled mesh campaign with 5 repetitions can rank them.
- **Forecasting again helped every policy:** median +41 % (v2), +42 % (old v3), +35 % (v3 + forecasting).

Files: `experiments/results/hardware/vid_campaign_wifi_fc/` ,

`vid_campaign_wifi_fc_summary.json/.csv/.log`. The forecaster costs 3.9 ms per arriving result (100 boxes) and 0.06 ms per displayed frame on the Pi 5 (measured).

Controller rebuilt for forecaster v2, with the forecaster running live: 80-run campaign

What ran. v3 rebuilt on the decay curve of the camera-aligned forecaster ("v3 + forecasting v2", `--decay staleness_decay_forecast_v2.json`) was compared with v2, fixed laptop YOLOv8m and v3 + forecasting v1. Setup: 4 scenarios × 5 repetitions × 4 policies = 80 runs, shuffled, on the shared Wi-Fi, 16:14–21:37 on 6 October. In **every** run the forecaster ran live in the Pi's camera loop: camera motion every frame, velocities on every arriving result, the held result forecast every frame. The extra load on the Pi was therefore the same for all policies. Each run saved what the Pi computed, and the scores below use the Pi's live camera motion.

Interruption. The Pi's Wi-Fi dropped at about 17:55 (the board did not reboot). The outage guard paused the campaign until it came back at 19:31. One run in progress was lost and rerun at the end, so all 80 are complete. Wi-Fi power saving on the Pi, which was on, was then switched off (runtime setting only).

SCENARIO	V3 + FORECASTING V2	V3 + FORECASTING V1	V2	FIXED LAPTOP YOLOV8M
Ground fails first	**0.0434 ± 0.0060**	0.0428 ± 0.0051	0.0342 ± 0.0098	0.0188 ± 0.0056
Jetson fails first	**0.0529 ± 0.0062**	0.0494 ± 0.0115	0.0475 ± 0.0140	0.0506 ± 0.0045
Random	0.0304 ± 0.0104	**0.0357 ± 0.0046**	0.0332 ± 0.0137	0.0257 ± 0.0161
No impairment	**0.0766 ± 0.0325**	0.0607 ± 0.0455	0.0701 ± 0.0367	0.0684 ± 0.0479
Availability (range over scenarios)	**0.995–0.998**	0.995–0.997	0.805–0.914	0.177–0.778

Streaming AP with forecaster v2 (quality-aware holding with the v2 decay curve), mean ± 95 % interval over 5 repetitions.

v3 + forecasting v2 beats v2, and the difference is significant

Paired over 20 runs: +0.0045 ± 0.0035 sAP (the interval excludes zero), median +8.9 %, ahead in 13 of 20. With the newest-result rule: +0.0038 ± 0.0037, ahead in 14 of 20. It is best in 3 of 4 scenarios. **VALIDATED** (Wi-Fi, 5 repetitions)

It closes the clean-link gap

With the earlier controllers, fixed laptop YOLOv8m won on a clean link (Part 1.4 above). Rebuilt on

With the earlier controllers, fixed laptop YOLOv8m won on a clean link (Part 14 above). Rebuilt on the v2 curve, v3 now leads there too (0.0766 against 0.0684). The objective now values accuracy correctly.

It keeps a fresh detection 99.5–99.8 % of the time

That holds in every scenario, against 81–91 % for v2 and 18–78 % for fixed laptop.

The live forecaster gives exactly the offline result

Scores using the camera motion computed live on the Pi are identical to the offline ones. The live forecaster ran over 80 runs with 0 errors.

v2 still beats the fixed laptop

+0.0054 ± 0.0054 (at the edge of significance), ahead in 15 of 20.

Cost found: onboard inference slows down

Camera tracking and the forecaster share the Pi's 4 CPU cores with onboard inference. Median YOLOv8s-320 time rose from 170 to 412 ms and YOLOv8n-320 from 93 to 110 ms, which weakens the onboard fallback in blackouts. Tracking every other frame, or at a lower resolution, is the obvious fix; not yet tested.

Files: `experiments/results/hardware/vid_campaign_wifi_fc2/` (runs with `_livefc.npz`), `vid_campaign_wifi_fc2_summary_v2_live.json/.log` (live camera motion), `vid_campaign_wifi_fc2_summary_v2.*` (offline), log `vid_campaign_wifi_fc2.log`.

PART 15

Phase G: energy, power and heat

Why this part exists. Until now the controllers optimized accuracy and freshness only. Energy was reported afterwards as a rough estimate (v1 had a tiny energy tie-breaker; v2 and v3 had none). This part measures energy and heat properly on both boards, rescores **every** earlier campaign with the measured model, and adds an energy- and heat-aware controller (v4).

G1. What was measured

Script `validation/hardware/energy_profile.py`, run on each board. Whole-board power was sampled 10 times a second from the Pi's power-management chip (all rails) and the Jetson's INA3221 input sensor. SoC temperature and CPU clock were logged alongside. Each state ran for 20 s. Inference ran back to back, with the same thread counts as the controller.

ran back to back, with the same thread counts as the controller.

STATE	PI 5 POWER	JETSON POWER	ENERGY PER FR
Idle (Jetson with clocks locked, as in all campaigns)	1.89 W	4.08 W	
JPEG encoding, 30 fps (640 q90 / 640 q50 / 320 q50)	+0.16–0.21 W	—	
YOLOv8n 320 onboard	6.0 W, 78 ms	5.7 W, 25 ms	Pi 324 m
YOLOv8s 320 onboard	6.1 W, 176 ms	6.8 W, 25 ms	Pi 741 m
YOLOv8s 640 onboard	5.0 W, 718 ms (throttled)	9.7 W, 29 ms	Pi 2,229 mJ
YOLOv8m 640	—	10.6 W, 57 ms	
Radio transmit at 2 / 5 / 10 Mbit/s; flat out	+0.27 / +0.30 / +0.33 W; +1.05 W at 52 Mbit/s	+0.09 W at 5 Mbit/s	≈ 0.2 W + 0.016 W per Mb
Live motion forecaster (camera tracking every frame + velocity updates)	+0.29 W (6-min steady test)	—	

The Pi PMIC rails exclude 5 V conversion losses, so Pi numbers are a lower bound. A short forecaster measurement taken right after a hot inference state read +0.67 W: hot silicon leaks more power. The 6-minute steady measurement (+0.29 W) is used.

On the Pi, thinking costs far more than talking

One YOLOv8s-640 inference costs 2.2 J; sending the same frame as a JPEG costs about 0.03 J (encode plus radio), roughly

80× less

. Onboard inference roughly triples the Pi's power (1.9 → 5–6 W).

The Jetson is about 14× more energy-efficient per inference than the Pi

0.16 J against 2.2 J for YOLOv8s-640. But it idles at 4.1 W even when doing nothing, so as a flying edge drone it is a large fixed cost.

G2. Heat: how much the Pi can sustain

Script `validation/hardware/thermal_steady.py` : constant load levels held for 6 minutes each, steady temperature taken over the last minute, actual ARM clock read from the firmware.

SUSTAINED LOAD	MEAN POWER	STEADY TEMPERATURE	ARM CLOCK (LAST MINUTE)
Idle	1.66 W	54 °C	1,500 MHz
JPEG encoding 30 fps	1.86 W	59 °C	1,670 MHz
Live forecaster 30 fps	1.95 W	60 °C	1,769 MHz
YOLOv8n onboard 15 % of the time	2.71 W	68 °C	2,291 MHz
YOLOv8n onboard 30 % of the time	3.49 W	76 °C	2,365 MHz
YOLOv8n onboard 50 % of the time	3.69 W	84 °C	**2,070 MHz (throttling)**
YOLOv8n onboard all the time	4.15 W	85 °C	**1,568 MHz (–35 %)**

+12.7 °C per extra watt; throttling from about 3.6 W sustained

A linear fit of the unthrottled levels gives T = 34.4 °C + 12.7 °C/W × P. The firmware starts lowering the clock at about 80 °C (about 3.6 W) and holds 85 °C at about 4 W by cutting the clock by 35 %. That is why YOLOv8s slowed from 576 to 718–758 ms in earlier tests. Heating is fast: 72 → 85 °C in 17 s under YOLOv8s. Cooling takes minutes (87 → 64 °C in 150 s).

The Jetson does not overheat

With its fan it peaked at 60 °C under YOLOv8m.

Energy and accuracy are coupled through heat

More onboard work → hotter → throttled → slower inference → staler results → lower accuracy. The fanless Pi cannot sustain the onboard load the adaptive controllers put on it.

G3. The energy model and flight time

`validation/hardware/energy_model.py` turns the measurements into energy per run:

- **Camera drone:** Pi idle × duration, plus each onboard inference's measured energy, plus each JPEG's encoding, plus radio energy for its bytes at the link speed measured in that run, plus the forecaster's power when forecasting is used.
- **Edge drone:** the Jetson's idle power plus every inference it served plus receiving. It counts only when a strategy uses it; the laptop is on the ground and not counted.
- **Flight time:** usable battery ÷ (flight power + perception power), for three drone classes:

DRONE CLASS	FLIGHT POWER	USABLE BATTERY	SOURCE
249 g mini drone	33.5 W	17.3 Wh	DJI Mini 2 published spec (2,250 mAh × 7.7 V, 31 min)
1.2 kg quadcopter	110 W	59 Wh	Paper's hover model; 4S 5,000 mAh, 80 % usable
1.5 kg drone carrying the Jetson	154 W	89 Wh	Same hover model; 6S 5,000 mAh, 80 % usable

What this means. Perception is 2–5 W against 33–154 W of flight power. On a 1.2 kg drone the choice of strategy changes flight time by only about 2–3 %; on a 249 g drone by up to about 9 %. Energy matters most through **heat** (throttling), through the **edge drone's** 4–6 W, and for **small drones**. The metric that combines everything is **mission perception** = streaming AP × flight time ("accuracy-minutes").

G4. Every placement with energy (Part 7 rescored)

PLACEMENT	ON TIME (MODEL)	DRONE ENERGY PER FRAME
Pi alone (YOLOv8s)	0	2,046 mJ
Pi split at layer 4 → Jetson (PDF's mothership)	0	999 mJ
Pi split at layer 4 → laptop	0	743 mJ
Pi JPEG → Jetson processes	17.0 /s	317 mJ

Jetson alone (camera on the Jetson)	15.0 /s	254 mJ
Jetson JPEG → laptop	14.4 /s	44 mJ
Pi JPEG → Jetson relay → laptop	14.2 /s	39 mJ
Pi JPEG → laptop	**30.0 /s**	**28 mJ**

Energy above idle on the drone-carried devices (Pi incl. radio; Jetson when it flies), from the Part 6 measured powers. Frame rates are the placement model's; live runs were lower (Part 7).

The professor's sense → edge → cloud pipeline (mothership split) costs about 35× more drone energy per frame than sending a JPEG to the ground, and is never on time. Splitting is worst on every axis: speed, accuracy and energy.

G5. Controllers v1 and v2 rescored (Parts 9–10)

POLICY (V2 CAMPAIGN, MEAN OVER 4 SCENARIOS)	ACCURACY-WEIGHTED ON-TIME /S	PI POWER	EDGE DRONE	PER PI WATT	STEADY PI TEMI
v2 (A_full)	**22.4**	4.05 W	5.0 W	5.5	≈ 86 °C →
All servers, no adaptation	19.1	5.11 W	5.3 W	3.7	woul
v2 without onboard (A_paths)	18.2	2.66 W	4.9 W	**6.9**	
Fixed laptop JPEG q50 640	13.7	2.16 W	—	6.4	
Fixed onboard YOLOv8n	6.2	5.08 W	—	1.2	woul

In v1, the adaptive controller ran at 2.95 W against 2.35–2.68 W for fixed JPEG offloading; the layer-1 split (5.6 W) and onboard-only (4.8 W) were worst. **In both campaigns the most energy-efficient adaptive policy is the one that does not use onboard inference.**

G6. Task-level campaigns with energy (Parts 12 and 14)

POLICY (80-RUN CAMPAIGN, POOLED)	SAP (FORECASTER V2)	FRESH-DETECTION SHARE	PI POWER	STEADY PI TEMP	EDGE DRON
v3 + forecasting v2	**0.0508**	**99.6 %**	4.17 W	≈ 87 °C, throttles	4.8
v3 + forecasting v1	0.0472	99.6 %	4.28 W	≈ 89 °C	4.9
v2	0.0463	85 %	4.38 W	≈ 90 °C	4.9
Fixed laptop YOLOv8m	0.0409	43 %	**2.41 W**	65 °C	

Pi power includes the forecaster. Mission perception = sAP × flight minutes of a 249 g drone. In the 108-run campaign the same pattern holds: adaptive policies 4.7–5.3 W, fixed offloading 2.4–2.6 W, onboard only 5.2 W; fixed laptop YOLOv8m and single-path rate adaptation have about 2× the accuracy per watt of the adaptive policies.

Our best controller wins on accuracy, freshness and mission perception, but not on energy

It uses about 1.8 W more on the Pi than fixed offloading, would throttle in sustained flight, and keeps a 4.8 W edge drone working. On a 249 g drone its accuracy gain outweighs its 1.3 minutes shorter flight (mission perception 1.41 against 1.18).

Onboard inference is the energy problem: about 40–54 % of the Pi's power for 3–5 % of the accuracy

Rescoring the adaptive runs with their onboard results removed (an offline estimate that ignores scheduling side effects) cuts Pi power from 4.2–5.3 W to about 2.4–2.5 W. Accuracy falls by only a median 2.7–5.3 %. For "all servers, no adaptation" accuracy even

rises

by 5.8 %: its low-quality onboard results displace better ones.

The earlier energy estimates were far too low

The old in-run estimate (onboard time × 2.7 W + transmit time × 0.31 W) left out the idle floor, encoding and the forecaster. In the same v4 test run it said 1.22 W where the board measured 3.56 W. All energy numbers in this part replace those estimates.

G7. Swarms with energy

- **Two real cameras (Part 13).** The Pi camera runs at 4.2–5.0 W in the adaptive settings, about 89–98 °C steady, so it would throttle; with fixed offloading it runs at 2.3–2.5 W. The Jetson as a camera draws about 7.4 W with adaptive control (mostly onboard GPU inference) and 4.2 W with fixed offloading. Serving the Pi adds about 1 W, so the Jetson board totals about 8.5 W.
- **Calibrated simulation with measured board powers** (`swarm_calibrated.py --measured-power`):
  - In large Pi swarms the coordinator lowers per-drone power: 4.5 → 3.5 W at 10 drones and 2.7 W at 20, because drones waiting their turn skip work, while delivering more.
  - Jetson-class drones under congestion shift to onboard GPU inference, so per-drone power rises from 4.6 to 6.9 W at 20 drones. The coordinator trims that slightly while delivering 21 % more.
  - Caveat: the simulator charges JPEG preparation at full compute power, scaled for the Pi's slow CPU, so its offloading energy is pessimistic against the measured +0.2 W.

G8. Controller v4: energy- and heat-aware

Objective. For every candidate (server, setting): v3's value minus μ × the candidate's **measured** energy per frame. The energy is the onboard inference energy, or encoding plus radio at the current link speed (`energy_constants.json`).

The price μ is not hand-tuned. It follows the Pi's **live measured** power, sampled 5 times a second, against a power budget by dual ascent: every second, $\mu \leftarrow \max(0, \mu + \eta(\bar{P} - \text{budget}))$, with $\eta = 0.1$. This keeps the average power at the budget, the standard way to enforce an average constraint online.

Budgets. 3.5 W (thermal-safe, below the throttling region from G2) and 2.8 W (energy-saving, about 70 °C).

Guards.

- Above 78 °C, μ jumps to at least 1, which makes onboard inference expensive.
- When no link can deliver, onboard is not priced (unless the Pi is at 85 °C), because it is then the only source of perception.

Every run of the new campaign logs measured power, temperature and μ. Code: `--energy-aware --power-budget` in `adaptive2.py` ; with the options off the controller is unchanged.

Smoke test. μ rose when the Pi hit 5.1 W, onboard inference dropped to a few frames in good phases and stayed on in the total blackout (82 frames), and the measured mean power was 3.56 W against a 3.5 W budget.

G9. Energy campaign: v4 against the best controllers, with measured energy

Setup. v4 at a 3.5 W and a 2.8 W budget against v3 + forecasting v2, v2 and fixed laptop YOLOv8m. 4 scenarios × 5 repetitions = 100 runs, shuffled, on the shared Wi-Fi, 14:53–19:19 on 7 October, no outages, all 100 complete. Every run logged the Pi's measured power, temperature and (for v4) the

energy price, 5 times a second, and ran the forecaster live. Accuracy is streaming AP with forecaster v2, using the live camera motion.

POLICY (POOLED, 20 RUNS EACH)	SAP	MEASURED PI POWER	SAP PER PI WATT	FRESH-DETECTION SHARE	MEAN
v4, 2.8 W budget	**0.0543 ± 0.0069**	3.33 ± 0.11 W	0.0166	87.1 %	
v4, 3.5 W budget	0.0525 ± 0.0072	3.61 ± 0.10 W	0.0147	86.8 %	
v3 + forecasting v2	0.0470 ± 0.0080	4.47 ± 0.17 W	0.0109	**99.3 %**	
v2	0.0465 ± 0.0067	4.24 ± 0.14 W	0.0113	88.7 %	
Fixed laptop YOLOv8m	0.0463 ± 0.0105	**2.65 ± 0.07 W**	**0.0171**	51.1 %	

Mission perception = sAP × flight minutes. The adaptive policies also keep the Jetson edge drone busy at about 4.8–4.9 W (model-based; the Jetson was not power-logged live); the fixed laptop policy does not use it.

PAIRED COMPARISON (20 PAIRS)	SAP DIFFERENCE	AHEAD IN	MEASURED PI POWER	MISSION PERCEPTION, 249 G
v4 (2.8 W) – v3 + forecasting v2	**+0.0074 ± 0.0028**	17 / 20	**−1.14 ± 0.19 W**	+0.245 ± 0.081
v4 (2.8 W) – v2	**+0.0078 ± 0.0034**	18 / 20	−0.91 ± 0.13 W	+0.250 ± 0.097
v4 (2.8 W) – fixed laptop YOLOv8m	**+0.0080 ± 0.0052**	16 / 20	+0.68 ± 0.15 W	+0.205 ± 0.148
v4 (3.5 W) – v3 + forecasting v2	+0.0055 ± 0.0043	15 / 20	−0.87 ± 0.18 W	+0.182 ± 0.123
v4 (2.8 W) – v4 (3.5 W)	+0.0018 ± 0.0031	10 / 20	−0.28 ± 0.09 W	+0.063 ± 0.086

The energy-aware controller is the most accurate controller, and uses 19–25 % less Pi power

v4 with a 2.8 W budget beats v3 + forecasting v2 by +0.0074 ± 0.0028 sAP (median +13 %, 17 of 20) and v2 by +0.0078 ± 0.0034, both significant, while measured Pi power falls by about 1 W. It

also has the highest mission perception for both drone classes. `VALIDATED` (Wi-Fi, 5 repetitions, measured energy)

Why saving energy raised accuracy

The onboard inferences v4 cuts were mostly low-quality (YOLOv8n/s at 320 px on aerial video) and competed with better offloaded results. They also loaded the Pi's 4 CPU cores and heat: with them, the Pi spent 76 % of the run at ≥ 80 °C, in the throttling region, slowing encoding, the camera loop and the forecaster for every result. v4 cuts that to 17 %. Energy, heat and accuracy are coupled, and pricing energy properly improves all three.

It nearly matches the most frugal strategy's accuracy per watt while being far more accurate

0.0166 sAP per watt against 0.0171 for fixed laptop offloading, with +0.0080 sAP (significant) and 87 % against 51 % fresh-detection share.

The price: freshness

v4 keeps a detection under 300 ms old 87 % of the time against 99 % for v3 + forecasting v2. Onboard results were what filled short gaps. A mission that needs a continuous fresh detection should use v3 + forecasting v2 or a higher budget.

The 2.8 W budget is below what v4 can reach

Measured 3.33 W. The Pi's fixed floor (idle, forecaster, encoding, radio and the power logger itself) is already about 2.65 W, and v4 deliberately keeps onboard inference during blackouts. The 3.5 W budget was held (3.61 W).

The energy model matches the measurements

Model-predicted Pi power is within 0.1–0.4 W (about 5–10 %) of the live-measured power in all five policies.

Files: `experiments/results/hardware/vid_campaign_energy/` (runs with `measured_energy` logs), `vid_campaign_energy_summary.log`, `energy/energy_campaign_measured.json`, `energy/energy_campaign_summary.json`; profiles and constants in `energy/`.

PART 16

What it all means: the argument for the paper

This part ties the results into one line of reasoning: what can be claimed, why the evidence supports it,

and where it stops.

1. Where to compute: send the image, don't split the network

Claim. For YOLO-class detectors on drone hardware, offloading a compressed image beats every split of the network.
Why. A split has to send intermediate features. Even INT8+zstd, the smallest split (k4, 310.7 KiB), is 2.0–11.5× a JPEG frame across 19 detectors. The drone also has to compute the first layers, which on a Pi takes 210–252 ms. Both costs push latency up, and they buy no accuracy, because JPEG q90 is near-lossless for detection.
Evidence. A closed-form bound, simulation, all 1,150 placements on real hardware, live runs (best split 7.0 frames/s against 15.2 for JPEG), and the calibrated swarm study (the split never beats JPEG in any cell).
Limit. A learned, trained compressor for the features could change this; that test is still to do.

2. How to schedule: use every computer, adapt per frame

Claim. A scheduler that keeps the onboard CPU, the edge GPU and the ground GPU busy in parallel and picks model, resolution and JPEG quality per frame gives the best perception when links fail.
Why. Each tier fails differently: the ground link drops out, the drone-to-drone link congests, and onboard is slow but never disconnected. Running them in parallel turns the failure of one into a lower rate rather than an outage. Adapting the setting to each path's current speed keeps results on time.
Evidence. On the proxy metric, +59–62 % over the best fixed strategy (88 runs). On real streaming accuracy over real Wi-Fi, v2 is ahead of every other policy on average, by +4.5 % up to +201 % (108 runs). This is significant against onboard, the Jetson and rule-based fallback.
Limit. Against all-servers-without-adaptation, single-path rate adaptation and the fixed laptop, the intervals still cross zero with 3 repetitions. Parallel offloading, adaptive device-plus-server use and per-frame adaptation are each known (Elf, Odessa, Chameleon). The contribution is the measured evidence on real drone tiers with streaming AP, not the mechanism.

3. What quality means on a drone: staleness, and how to undo it

Claim. On aerial video a detection loses most of its value within 100–200 ms, and motion forecasting recovers much of it cheaply.
Why. The camera platform itself moves, so even static objects shift across the image. A 300 ms "on-time" deadline counts a result as fully useful when it has already lost over 90 % of its accuracy. Forecasting moves each box along its observed motion, which is mostly smooth over a few hundred milliseconds.
Evidence. The decay is measured twice (GPU and CPU detections agree within 1 point). Forecasting raises kept accuracy at 267 ms from 7 % to 47 % on training videos, and raises streaming AP for all 9 policies on the 108 test runs (median +32 % to +67 %), with settings fixed before the test. On the Pi it costs 3.9 ms per arriving result.

Stronger version. Measuring the camera's own motion from the images (2.7 ms per frame on the Pi) and forecasting only the objects' residual motion keeps 60 % of accuracy at 267 ms, and roughly doubles streaming AP on the real runs. It beats the simple forecaster in all 144 runs.
Prior work. Tracking to hide offloading delay (Glimpse 2015; Liu et al., MobiCom 2019), latency forecasting (streaming perception; PVT++ on UAVs) and optical-flow camera-motion compensation (BoT-SORT) are all established. What is ours: the measured aerial decay curves with and without correction, and the measured gains on a real distributed drone system.
Limit. Constant velocity per object, zero velocity for newly seen objects, and a similarity transform for the camera (no perspective). An IMU-assisted or learned forecaster may add more.

4. Scheduling for the decay you actually have

Claim. The scheduler's objective must use the decay curve the drone really experiences. With forecasting that curve is far flatter, and a scheduler built on it keeps a fresh detection almost all the time.
Why. Built on the steep curve, v3 chased the fastest results and lost accuracy (–14 % against v2). Rebuilt on the forecasting curve, it values steady, slightly older results correctly.
Evidence. In a paired 36-run campaign, v3 + forecasting held a detection under 300 ms old 99.0–99.9 % of the time in every scenario (v2: 75–92 % under impairment), with accuracy level with v2.
Limit. Accuracy ranking against v2 is inconclusive with 3 repetitions on a shared Wi-Fi, where session-to-session variation is as large as the gap between controllers.
Further support. With the stronger forecaster v2 the decay is flatter still, and the policy that wins on a clean link changes to the big accurate model. Every controller built for a steeper curve under-uses it.
Confirmed. v3 rebuilt on the v2 curve, with the forecaster running live on the Pi, beats v2 significantly over 20 paired runs (+0.0045 ± 0.0035 sAP) and wins the clean link too. This is the clearest evidence for the principle: match the objective to the forecaster in use.

5. Many drones: decide together, not alone

Claim. When drones share a network, per-drone adaptation is not enough; a light admission coordinator prevents collapse and starvation.
Why. Each drone sees only its own link and plans as if it had a fair share. Together they overload the network: frames arrive late everywhere, or the router serves some drones and starves others. Limiting how many offload at once, and rotating turns, keeps the shared link inside its capacity.
Evidence. Calibrated simulation over 96 cases: better in 16, tied in 74, worse in 6. Same split on held-out seeds. 20 Pi drones go from 0 to 4.15 detections/s each, and the worst-served drone from 0 to 7.65.
Stronger version. With each drone's radio reporting its own link health, the coordinator stops mistaking a drone's own blackout for congestion or starvation. The one real loss disappears (–7.4 % → +0.5 %), and every remaining loss is at most 1.9 %, on held-out seeds too.
Credibility. The simulator, rebuilt from 44 real runs, predicts on-time rates per phase with correlation 0.91.
Hardware. With two real camera drones, sharing the ground station costs the Pi 11–19 % of its accuracy, and adaptive control beats fixed offloading in 6 of 6 pairs. The coordinator rarely acts with two

accuracy, and adaptive control beats fixed offloading in 6 of 6 pairs. The coordinator rarely acts with two cameras, as the simulation predicts for small swarms.

Prior work. Splitting a shared uplink among cameras with per-camera adaptation is established (JCAB, INFOCOM 2020). Ours is the admission rule with radio link health and its hardware calibration.

Limit. The coordinator's gains are still simulation results. The simulator misses the slow recovery after blackouts. The extra boards will allow 3–5 real cameras, where the gains should appear.

6. Energy, heat and accuracy are coupled: price energy, and accuracy rises

Claim. On a small fanless drone computer, energy-aware scheduling is not a trade-off against accuracy; done right, it improves it.

Why. Onboard inference costs about 80× the energy of sending a JPEG, heats the Pi into throttling within seconds, and steals CPU from encoding, the camera loop and forecasting. Pricing energy against a measured power budget (dual ascent on live power) removes the low-value onboard work and keeps the board out of the throttling region.

Evidence. Measured power and temperature on both boards. Every earlier campaign was rescored. In a 100-run campaign v4 cut measured Pi power by about 1 W and throttling-region time from 76 % to 17 %, while being significantly more accurate than every other controller (+0.0074 ± 0.0028 sAP over v3 + forecasting v2). It also has the highest mission perception for 249 g and 1.2 kg drones.

Prior work. Energy-aware offloading for drones and mobile devices is a large field, and DVFS and thermal-aware scheduling are known. What needs checking: whether the measured coupling, where energy pricing *raises* streaming accuracy through heat and CPU contention, has been shown before.

Limit. Freshness falls from 99 % to 87 %. The Jetson's power was modelled, not logged live. Runs last 2 minutes; longer flights would heat the Pi more, which should favour v4 further.

Honest negatives, also part of the paper

- The original staleness-aware v3 did not beat v2.
- Feedback learning in the coordinator did not pay off overall.
- On a clean link, a fixed strong server is as good as any adaptive policy.
- The PDF's mothership split design was the worst placement measured (Part 8).

PART 17

Faults found and fixed

FAULT	EFFECT	FIX
Ultralytics caps PyTorch at 1 thread	Pi 65 % slower	Threads reset after import
Jetson GPU down-clocks at low load; reboot clears the lock	Remote times doubled	`jetson_clocks` before every run

Frames queued in the kernel send buffer	1.3 s latency	Send the next frame only after the previous one has left
Old result files read as new	Stale live numbers	Delete before each run
Onboard scored below sending to a dead link	No blackout fallback	Late results keep 25 % value; energy weight reduced
TCP back-off after blackouts	10+ s recovery	Reconnect stalled links; 2.5 s worker liveness timeout
Stall rule judged total transfer time	Large payloads penalized	Stall = no acknowledged bytes for 1.5 s
Leftover workers from earlier runs	Jetson path crippled (27 runs discarded)	Kill stale processes before every run
Sender thread died on a closed socket	Link dead for the rest of a run (10 runs rerun)	Closed socket treated as a failed link; thread cannot die
Slow NMS at confidence 0.01	Up to 130 ms per frame	Vectorized GPU NMS, same AP
Stuck remote-time estimates after warm-up	Servers abandoned	Warm-up at all sizes; stale estimates relax toward the measured prior
Temp-file cleanup wiped the helper scripts	Lost tools	Helpers moved into the repo
Network outages during campaigns	Runs failing in a loop	Outage guard pauses until both boards respond

PART 18

Code status and reproducibility

State on 6 October 2026, 15:25, after all campaigns finished:

Nothing is running

All campaigns and simulations have finished; the Pi and the Jetson are idle.

All 118 project tests pass

`python -m pytest tests` , 17 s, including the simulator tests. This matters because `simulation/stream_sim.py` gained the coordinator. **VALIDATED**

The simulator's default behaviour is unchanged

Three swarm cells computed before the coordinator existed (Pi adaptive 5 drones, Jetson JPEG 10 drones, Jetson adaptive 20 drones) were recomputed with the new code. They are bit-for-bit identical. The coordinator and its feedback layer only act when switched on (`coordination="admission"` , `coord_feedback`), so every earlier paper and simulation result stays valid. **VALIDATED**

Every new or changed script compiles

`forecast.py` , `stream_ap.py` , `summarize_vid.py` , `swarm_calibrated.py` , `adaptive2.py` , `stream_sim.py` , plus a syntax check of `campaign_vid.sh` .

Not committed yet

`simulation/stream_sim.py` is the only modified tracked file (+124 lines). `validation/hardware/` , the new docs and `experiments/results/hardware/` are untracked. Ready to commit on `publication-overhaul` when you say so.

PIECE	FILE	STATE
Swarm coordinator and feedback ablation	`simulation/stream_sim.py`	Working; confirmed on held-out seeds
Motion forecaster	`validation/hardware/forecast.py`	Working; timed on the Pi
Streaming-AP scoring with forecasting	`stream_ap.py` , `summarize_vid.py`	Working; scored all 144 Wi-Fi runs
Calibrated swarm study	`validation/hardware/swarm_calibrated.py`	Working; resumable per-cell cache
Campaign runner	`validation/hardware/tools/campaign_vid.sh`	Working; policy subsets, heartbeat log, result recovery, outage guard; 144 runs without failures

How to reproduce

RESULT	COMMAND (FROM `UAV-SPLIT-PERCEPTION/` , VENV ACTIVE)
Swarm study, with held-out seeds	`python -m validation.hardware.swarm_calibrated` ; then `--seed0 5`
Forecaster: detections, tuning, decay	`cd validation/hardware; python forecast.py dets; python forecast.py tune; python forecast.py decay`
Campaign statistics	`python summarize_vid.py <dir>`

Campaign statistics	`python summarize_vid.py --dir ../../experiments/results/hardware/vid_campaign_wifi --ref A_full`
A hardware campaign	`OUTDIR=… REPS=3 ONLY="A_stream_fc A_full A_stream" validation/hardware/tools/campaign_vid.sh` (board passwords from environment variables only)

PART 19

What is left

TASK	NEEDS	TIME
Move the three devices onto AegisSwarm; laptop internet over Ethernet; measure each mesh link	Your cable and mesh details	1–2 h
v4 refinements: recover freshness (allow cheap onboard results only when the newest detection is about to go stale); live Jetson power logging; 20–30 min runs to show sustained heating	Code + one campaign night	1–2 days
Make the live forecaster cheaper on the Pi (camera tracking every other frame or at lower resolution)	Code + short campaign	half a day
Task-level campaign on the mesh: the 9 policies plus v3 + forecasting v1 and v2, 4 scenarios × 5 repetitions (220 runs). Settles the accuracy ranking of v2 against v3 + forecasting	Mesh set up; unattended	about 9–10 h
Swarm test with 3 Pis and 2 Jetsons (5 cameras) on the mesh	Your new boards set up	3–5 days
Learned-split test: can a trained compressed split beat JPEG?	The laptop GPU free	1–2 weeks, overnight
Commit the work to git (branch `publication-overhaul`)	Your go-ahead	minutes
Paper and professor report update	—	1–2 weeks

PART 20

Where everything is

PATH (IN `UAV-SPLIT-PERCEPTION/`)	CONTENTS
`paper/main.tex`	The paper (simulation study, theory, validation)

docs/hardware_results.md	Device, link and placement results
docs/adaptive_results.md	v2 scheduler results
docs/controller_model.md	Formal model of controller v3
docs/novelty_and_related_work.md	Literature check and claims
docs/assumptions_and_validation.md	Every model input and its validation status
validation/hardware/	All hardware scripts and the campaign tools
experiments/results/hardware/	Raw results; invalid campaigns kept in *_invalid_* folders
experiments/results/hardware/vid_campaign_wifi/	Real-world Wi-Fi campaign (log: vid_campaign_wifi.log)
experiments/results/hardware/swarm_calibrated.csv , _summary.json	Calibrated swarm simulation, with coordination (per-cell cache in swarm_calibrated_cache/ ; held-out seeds in swarm_calibrated_seeds5-9.*)
experiments/results/hardware/vid_campaign_wifi_fc/	Paired campaign: v3 + forecasting, v2, old v3 (log: vid_campaign_wifi_fc.log)
experiments/results/hardware/staleness_decay_forecast.json	Decay curve with and without forecasting (exact ages, train)
experiments/results/hardware/_superseded/ , swarm_calibrated_cache_superseded_coord_v1/	Files set aside, not deleted: copies wrongly pulled into a campaign folder, and simulation cells from an older coordinator
experiments/results/hardware/energy/ , vid_campaign_energy/	Energy profiles, thermal steady states, constants, rescoring of every campaign, the v4 energy campaign
validation/hardware/energy_profile.py , thermal_steady.py , energy_model.py , energy_rescore.py	Energy and heat measurement, model, rescoring
experiments/results/hardware/forecast_tuning_v2.json , staleness_decay_forecast_v2.json , *_summary_v2.*	Forecaster v2: tuning on train, decay curve, rescored campaigns
experiments/results/hardware/forecast_tuning.json	Forecaster tuning on train (all 128 settings) and the frozen setting

PART 21

Glossary

Cut k	Layers 0 to k run on the drone; the rest run on the server.
INT8 + zstd	Each number rounded to 8 bits, then losslessly compressed.
JPEG q90 / q50	JPEG quality 90 (high) or 50 (smaller, less accurate).
NMS	Non-maximum suppression: removes duplicate boxes after the network.
mAP	Mean average precision, the standard detection accuracy.
sAP	Streaming AP: accuracy of what the drone holds at each instant.
Availability	Share of time the newest detection is under 300 ms old.
netem	Linux tool used to impose rate limits and blackouts.
IBSS / batman-adv	Ad-hoc Wi-Fi without a router, and a mesh routing protocol on top of it.
Staleness / decay curve	How much accuracy a detection keeps as it ages; measured on training videos.
Motion forecasting	Moving each old box forward by its estimated velocity × its age.
Ego-motion	The camera's own motion between frames, here measured by tracking background points (optical flow) and fitting a rotation/scale/shift.
Admission control	Letting only as many drones offload as the shared network can carry.
Water-filling / capped sharing	Two models of how a network divides capacity: equal fair shares, or a scheduler that serves a few drones and starves the rest.
AIMD	Additive increase, multiplicative decrease: the TCP rule of cutting fast on trouble and growing back slowly.
Held-out seeds	Random seeds not used while designing, kept to confirm a result.
v1 / v2 / v3	The three adaptive controllers: single-path, multi-server, and staleness-aware.
Dual ascent	Adjusting a price (here, of energy) up when a constraint is exceeded and down when not, so the average meets a budget.
Throttling	The Pi's firmware lowering its clock to stay below about 85 °C, making everything slower.
Mission perception	Streaming AP × flight minutes: accuracy and endurance in one number.
v4	The energy- and heat-aware controller: v3 + forecasting v2 with an energy price held to a measured power budget.
Phase oracle	The best fixed strategy per phase, chosen with hindsight.